Mathematical Models of the Drone

A Learning Guide: every equation derived, explained
with toy examples, and connected to our drone

3D Mapping of an Environment using a Normal Monocular Camera
and Controlling the Drone by Visual-Inertial Odometry (VIO)

Model 1	Nonlinear dynamics → linearization → error-state EKF → geometric SO(3) control → motor allocation
Model 2	Flight data → DMDc system identification → Bellman / LQR / Riccati / DARE **and** MPC / QP → drone → EKF on the learned model (camera + gyro) → controllers

Group CD2

Ravishanmugam K ▪ Ishwarya Murugesan ▪ Aparna Bharani ▪ Cibikumar B ▪ Akhilan S

Amrita Vishwa Vidyapeetham

Who this is for. Anyone who wants to understand how our drone works: classmates, future students, or ourselves in a year. No prior knowledge of estimation or control is assumed. Start with section 1, which gives the big picture and a short maths refresher. Every toy example in this guide can be re-run in MATLAB with the scripts in `toy_model2/`, which print the same numbers.

Contents

1 Start Here **3**
1.1 The big picture in plain words . . . 3
1.2 How every block is explained . . . 3
1.3 Maths refresher . . . 3

2 Notation, Conventions and Tools **5**
2.1 Reference frames . . . 5
2.2 Typographic conventions . . . 6
2.3 Master symbol table . . . 6
2.4 The SO(3) toolkit . . . 7
2.5 What was corrected with respect to the original images . . . 8

3 The Two Equation Flows **8**

4 Model 1 — Estimation and Geometric Control **11**
4.1 Block 1 — Physical system and nonlinear dynamics . . . 11
4.2 Block 2 — Linearization: Taylor expansion and Jacobians . . . 13
4.3 Block 3 — Local error dynamics . . . 15
4.4 Block 4 — State estimation: error-state EKF . . . 16
4.5 Block 5 — Position control (outer loop) . . . 19
4.6 Block 6 — Force and attitude generation . . . 20
4.7 Block 7 — Attitude control on SO(3) (inner loop) . . . 21
4.8 Block 8 — Actuator commands and feedback . . . 23
4.9 Recap of Model 1 . . . 25

5 Model 2 — Data-Driven Model and Optimal Control **25**
5.1 Block 1 — Nonlinear flight data . . . 26
5.2 Block 2 — DMDc: Dynamic Mode Decomposition with control . . . 27
5.3 Block 3 — Bellman equation (optimal control) . . . 29
5.4 Block 4 — Linear Quadratic Regulator (LQR) . . . 30
5.5 Block 5 — Riccati recursion . . . 31
5.6 Block 6 — DARE: Discrete Algebraic Riccati Equation . . . 32
5.7 Block 7 — Model Predictive Control (MPC) . . . 33
5.8 Block 8 — Optimization formulation: Quadratic Program . . . 34
5.9 Block 9 — MPC control input (receding horizon) . . . 36
5.10 Block 10 — Apply to the drone . . . 37
5.11 Block 11 — State estimation: Extended Kalman Filter . . . 38
5.12 Recap of Model 2 . . . 42

6 How the Two Models Fit Together **42**

6.1 One drone, two ways of thinking 42

6.2 The complete picture 42

6.3 The toy model: `toy_model2` 43

A Proofs of the SO(3) identities **43**

B Matrix-calculus rules used **44**

C References **44**

D Glossary **45**

1 Start Here

This guide explains, from the ground up, all the mathematics our group used to make a drone know where it is and fly where we want it to. You do not need to have seen any of it before. If you can multiply a matrix by a vector and take a derivative, you can follow every page. The rest (Jacobians, covariance, eigenvalues and so on) is explained in section 1.3 with small examples.

1.1 The big picture in plain words

Flying a drone autonomously means answering four questions, again and again, many times per second.

Question	What answers it	Where
How does a drone move at all?	Physics: Newton's laws for a spinning, tilting body	Model 1, blocks 1–3
Where am I right now?	Combining camera and IMU with a model (an EKF; this is our "VIO")	M1 block 4, M2 block 11
Where should I go, and how do I get there?	A controller: geometric (Model 1) or optimal (Model 2)	M1 blocks 5–7, M2 blocks 3–9
How do I make the motors do it?	Splitting thrust and torque into four motor speeds	M1 block 8, M2 block 10

We worked through this in two different ways, which are the two flow diagrams of section 3.

- **Model 1** starts from *physics*. Write down the equations of motion, use them to estimate the state, and control it with a controller that works directly with rotations (no Euler angles).
- **Model 2** starts from *data*. Fly, record, let the computer find a linear model that explains the recording, and then compute the best possible controller for that model.

1.2 How every block is explained

Each block of both flow diagrams gets its own subsection, always in the same order, with the same colours:

Part	What it gives you
Coloured equation box	The equations exactly as in the flow diagram, with their number (M1.x, M2.x)
Symbol table	Every symbol: what it means, its size (dimension) and its unit
Derivation	Where the equation comes from, step by step, nothing skipped
Toy example (orange box)	Small numbers you can check with a calculator
In our drone project (green box)	What it means for our drone, in plain language
Toy model (grey box)	Which MATLAB script in `toy_model2/` runs it and prints the same numbers

Three ways to read this guide.

1. *Ten-minute tour:* this section, the two flow diagrams, and the recap tables at the end of each model (sections 4.9 and 5.12).
2. *Understanding:* read only the green "In our drone project" boxes and the toy examples, in order.
3. *Full learning:* everything in order, and run the matching MATLAB step next to each block.

1.3 Maths refresher

Only the tools that appear later are covered here, each with the smallest possible example.

Vectors, matrices, transpose. A vector $\boldsymbol{a} \in \mathbb{R}^3$ is a column of 3 numbers, and a matrix $M \in \mathbb{R}^{m\times n}$ has m rows and n columns. The **dimension** written next to every symbol in this guide is exactly this size. The transpose $M^\top$ swaps rows and columns, and $\boldsymbol{a}^\top \boldsymbol{b}$ is the dot product.

Toy example

$\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}1\\ 1\end{bmatrix} = \begin{bmatrix}3\\ 7\end{bmatrix}$, $[1,2,3]\,[4,5,6]^\top = 4+10+18 = 32$. A 4×12 matrix times a 12-vector gives a 4-vector.

Dots mean "rate of change". $\dot{\boldsymbol{p}} = d\boldsymbol{p}/dt$ is velocity and $\ddot{\boldsymbol{p}}$ is acceleration. An equation $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ is a *differential equation*: it tells you how fast the state changes. A computer steps it forward in small time steps Δt, e.g. $\boldsymbol{x}(t+\Delta t) \approx \boldsymbol{x}(t) + \Delta t\, \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ (Euler's method; we use the more accurate RK4).

Discrete time. Computers work in ticks $t_k = k\,\Delta t$. A subscript k means "at tick k", so $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ says: next state = matrix × current state + matrix × current input.

Partial derivatives and the Jacobian. $\partial f/\partial x$ is how much f changes when only x is nudged. For a vector function $\boldsymbol{f}$ of a vector $\boldsymbol{x}$, all these numbers form a matrix, the **Jacobian** $A = \partial \boldsymbol{f}/\partial \boldsymbol{x}$, where entry (i, j) is $\partial f_i/\partial x_j$. Near a point, the function behaves like the linear map $\delta \boldsymbol{f} \approx A\,\delta \boldsymbol{x}$. This is the multi-dimensional version of "slope".

Toy example

$\boldsymbol{f}(x, y) = \begin{bmatrix}x\,y\\ x + y^2\end{bmatrix}$ at $(x, y) = (2, 1)$: $A = \begin{bmatrix}y & x\\ 1 & 2y\end{bmatrix} = \begin{bmatrix}1 & 2\\ 1 & 2\end{bmatrix}$. Nudging (x, y) by $(0.1, 0)$ changes $\boldsymbol{f}$ by $\approx A[0.1, 0]^\top = [0.1, 0.1]^\top$. The exact change is $[0.1, 0.1]^\top$.

Linear versus nonlinear. A model is linear if doubling the input doubles the effect, as in $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$. Drones are nonlinear because of sin, cos and products such as $\boldsymbol{\omega} \times J\boldsymbol{\omega}$. Linear models are far easier to analyse, so we *linearize*: approximate the nonlinear model by its Jacobian near an operating point such as hover.

Mean, variance, covariance, Gaussian. The mean is the average value. The variance σ^2 is the average squared distance from the mean, and σ is the "typical error". For a vector of errors, the **covariance matrix** $P = \mathbb{E}[\boldsymbol{e}\boldsymbol{e}^\top]$ holds the variance of each component on its diagonal and how components move together off the diagonal. A Gaussian $\mathcal{N}(\mu, \sigma^2)$ is the bell curve: about 68% of samples lie within $\pm\sigma$ and 99.7% within $\pm 3\sigma$.

Toy example

Measurements 9.8, 10.1, 10.1: mean 10.0, deviations -0.2, 0.1, 0.1, variance $(0.04 + 0.01 + 0.01)/3 = 0.02$, $\sigma = 0.14$. A rule we use constantly: if $\boldsymbol{e}' = F\boldsymbol{e}$, then its covariance is $P' = FPF^\top$.

Minimising a function. At the lowest point of a smooth function the slope is zero. For a quadratic $g(u) = au^2 + bu + c$ with $a > 0$: $g'(u) = 2au + b = 0$ gives $u^* = -b/(2a)$. The matrix version, $g(\boldsymbol{u}) = \boldsymbol{u}^\top M\boldsymbol{u} + \boldsymbol{b}^\top \boldsymbol{u}$ with M positive definite, gives $\boldsymbol{u}^* = -\frac{1}{2}M^{-1}\boldsymbol{b}$. Both LQR and the Kalman gain are derived exactly this way.

Toy example

$g(u) = u^2 - 4u + 1$: $g'(u) = 2u - 4 = 0 \Rightarrow u^* = 2$, and $g(2) = -3$ is the minimum.

Positive definite. $M \succ 0$ means $\boldsymbol{x}^\top M \boldsymbol{x} > 0$ for every non-zero $\boldsymbol{x}$: a "bowl" that opens upwards in every direction. Cost weights Q and R, and covariances P, are of this kind.

Eigenvalues and stability. If $M\boldsymbol{v} = \lambda \boldsymbol{v}$, then $\boldsymbol{v}$ is a direction that M only stretches, by the factor λ. For $\boldsymbol{x}_{k+1} = M\boldsymbol{x}_k$, each such direction is multiplied by λ every tick:

$$|\lambda| < 1 \text{ for all eigenvalues } \Rightarrow \text{ the state dies out (stable),}$$
$$|\lambda| > 1 \text{ for at least one } \Rightarrow \text{ it blows up (unstable).}$$

Toy example

$x_{k+1} = 1.1\, x_k$: 1, 1.1, 1.21, 1.33, ... blows up. $x_{k+1} = 0.62\, x_k$: 1, 0.62, 0.38, 0.24, ... dies out. Section 5.4 uses exactly this pair: LQR turns the first into the second.

Rotations. The attitude R is a 3×3 matrix whose columns are the drone's own forward, left and up axes, written in world coordinates. It is *not* three angles. Section 2.4 gives all the tools needed to work with it (hat map, Exp, Log).

2 Notation, Conventions and Tools

Both models describe the same physical object, a quadrotor, so they share one notation. This section fixes it once. If a symbol appears later without explanation, it is defined here.

2.1 Reference frames

- **World frame** $\mathcal{W}$: East–North–Up (ENU), fixed to the ground. Its third axis $\boldsymbol{e}_3 = [0, 0, 1]^\top$ points *up*. ArduPilot's ROS 2 topics `/ap/v1/pose/filtered` and `/ap/v1/twist/filtered` are expressed in this frame.
- **Body frame** $\mathcal{B}$: Forward–Left–Up (FLU), fixed to the drone, origin at the centre of mass. The propellers push along the body z-axis.
- **Attitude** $R \in \mathrm{SO}(3)$ rotates body vectors into world vectors: $\boldsymbol{a}^{\mathcal{W}} = R\, \boldsymbol{a}^{\mathcal{B}}$. Its columns are the body axes written in the world frame, $R = [\boldsymbol{b}_1\ \boldsymbol{b}_2\ \boldsymbol{b}_3]$.

With ENU, gravity is the *vector* $\boldsymbol{g} = [0, 0, -g_0]^\top$ with $g_0 = 9.81\,\mathrm{m/s^2}$. This is why the desired force later reads $\boldsymbol{F}_d = m(\boldsymbol{a}_d - \boldsymbol{g})$: subtracting a downward vector adds an upward push.

2.2 Typographic conventions

Convention	Meaning
$\boldsymbol{p},\ \boldsymbol{v},\ \boldsymbol{\omega}$ (bold lower case)	column vectors
$R,\ J,\ A,\ P$ (upright capitals)	matrices
$\hat{(\cdot)}$	estimate produced by the EKF, e.g. $\hat{\boldsymbol{p}}$, $\hat{R}$
$(\cdot)_d$	desired / reference value, e.g. $\boldsymbol{p}_d$, R_d
$(\cdot)_k$	value at the sampling instant $t_k = k\,\Delta t$
$\delta(\cdot)$	small deviation (error) from an operating point
$[\boldsymbol{a}]_\times$	skew-symmetric ("hat") matrix of $\boldsymbol{a}$, see (1)
$(\cdot)^\vee$	inverse of the hat map
$\delta\boldsymbol{x}$	12-number error between two states, see (6); correcting by it: (7)
$Q \succeq 0,\ R \succ 0$	positive semi-definite / positive definite matrix
$X^\dagger$	Moore–Penrose pseudo-inverse

Two different "R". $R \in \mathrm{SO}(3)$ is the attitude. In Model 2 and in the EKF the same letter is the standard name of an input or noise weight. When both could appear together, the noise covariance is written R_n. In Model 2 the attitude only appears through $\mathrm{Log}(R_k)$, so "R" there always means the input weight.

2.3 Master symbol table

Symbol	Meaning	Dim.	Unit	Project value
$\boldsymbol{p}$	position of the centre of mass (world)	$\mathbb{R}^3$	m	–
$\boldsymbol{v}$	velocity (world)	$\mathbb{R}^3$	m/s	$\lvert v_i\rvert \le 1$ (`max_velocity`)
R	attitude, body $\to$ world	$\mathrm{SO}(3) \subset \mathbb{R}^{3\times3}$	–	–
$\boldsymbol{\omega}$	angular velocity (body)	$\mathbb{R}^3$	rad/s	–
T	total thrust along $\boldsymbol{b}_3$	$\mathbb{R}$	N	hover $mg_0 = 14.715$
$\boldsymbol{\tau}$	torque about body axes	$\mathbb{R}^3$	N m	–
$\boldsymbol{u} = [T;\boldsymbol{\tau}]$	control input (wrench)	$\mathbb{R}^4$	N, N m	–
m	mass	$\mathbb{R}$	kg	1.5
J	inertia matrix	$\mathbb{R}^{3\times3}$	kg m^2	$\mathrm{diag}(0.02, 0.02, 0.04)$
$\boldsymbol{g}$	gravity vector (ENU)	$\mathbb{R}^3$	m/s^2	$[0, 0, -9.81]^\top$
$K_p,\ K_v$	position / velocity gains	$\mathbb{R}^{3\times3}$	s^{-2}, s^{-1}	$\mathrm{diag}(1.5, 1.5, 2)$, $\mathrm{diag}(1.2, 1.2, 1.5)$
$K_R,\ K_\omega$	attitude / rate gains	$\mathbb{R}^{3\times3}$	N m, N m s	$\mathrm{diag}(4, 4, 2)$, $\mathrm{diag}(0.8, 0.8, 0.5)$
Ω_i	speed of rotor i	$\mathbb{R}$	rad/s	$100 \dots 1100$
k_f	rotor thrust constant, $f_i = k_f\Omega_i^2$	$\mathbb{R}$	N s^2	8.549×10^{-6}
c_m	yaw-torque-to-thrust ratio	$\mathbb{R}$	m	0.016
Δt	sample time	$\mathbb{R}$	s	0.01 (Model 1), 0.04 (Model 2)

The values marked as project values come from `drones_controller/config/controller.yaml` (mass, gravity, inertia, all gains, `max_velocity`, `control_rate`=25 Hz, the setpoint $\boldsymbol{p}_d = [0, 0, 2]$ m). The rotor constants and geometry are those of the ArduPilot Gazebo *Iris* quadrotor. It is the vehicle simulated in our SITL set-up and also weighs 1.5 kg.

2.4 The SO(3) toolkit

The attitude R is a 3×3 matrix with $R^\top R = I$ and $\det R = +1$. It has 9 entries but only 3 degrees of freedom, so it cannot be added or subtracted like a vector. The following tools are used throughout.

Hat and vee maps. For $\boldsymbol{a} = [a_1, a_2, a_3]^\top$,

$$[\boldsymbol{a}]_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \qquad [\boldsymbol{a}]_\times \boldsymbol{b} = \boldsymbol{a}\times\boldsymbol{b}, \qquad [\boldsymbol{a}]_\times^\vee = \boldsymbol{a}. \tag{1}$$

Toy example — hat map

$\boldsymbol{a} = [1,2,3]^\top$, $\boldsymbol{b} = [0,0,1]^\top$: $[\boldsymbol{a}]_\times \boldsymbol{b} = \begin{bmatrix} 0 & -3 & 2 \\ 3 & 0 & -1 \\ -2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \\ 0 \end{bmatrix} = \boldsymbol{a}\times\boldsymbol{b}.$

Useful identities (proved in section A):

$$[\boldsymbol{a}]_\times^\top = -[\boldsymbol{a}]_\times, \qquad [\boldsymbol{a}]_\times \boldsymbol{a} = \boldsymbol{0}, \qquad [\boldsymbol{a}]_\times \boldsymbol{b} = -[\boldsymbol{b}]_\times \boldsymbol{a}, \tag{2}$$

$$R\,[\boldsymbol{a}]_\times\,R^\top = [R\boldsymbol{a}]_\times \quad (R\in \mathrm{SO}(3)), \tag{3}$$

$$\operatorname{tr}(M[\boldsymbol{\eta}]_\times) = -\boldsymbol{\eta}^\top (M - M^\top)^\vee \quad \text{for any } M\in\mathbb{R}^{3\times3}. \tag{4}$$

Exponential and logarithm. A rotation by angle θ about the unit axis $\boldsymbol{n}$ is

$$\operatorname{Exp}(\boldsymbol{\phi}) = I + \sin\theta\,[\boldsymbol{n}]_\times + (1-\cos\theta)\,[\boldsymbol{n}]_\times^2, \qquad \boldsymbol{\phi} = \theta\boldsymbol{n}\in\mathbb{R}^3 \tag{5}$$

(Rodrigues' formula), and $\operatorname{Log} : \mathrm{SO}(3) \to \mathbb{R}^3$ is its inverse, $\operatorname{Log}(R) = \frac{\theta}{2\sin\theta}(R - R^\top)^\vee$ with $\theta = \arccos\frac{\operatorname{tr}R-1}{2}$. For small $\boldsymbol{\phi}$: $\operatorname{Exp}(\boldsymbol{\phi}) \approx I + [\boldsymbol{\phi}]_\times$.

Toy example — 10° roll

$\boldsymbol{\phi} = [0.1745, 0, 0]^\top$ (10° about x) gives $\operatorname{Exp}(\boldsymbol{\phi}) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.9848 & -0.1736 \\ 0 & 0.1736 & 0.9848 \end{bmatrix}$. The small-angle form $I + [\boldsymbol{\phi}]_\times$ has ±0.1745 instead of ±0.1736 off the diagonal and 1 instead of 0.9848 on it, an error below 2%.

Errors and corrections when the state contains R. The full state $\boldsymbol{x} = (\boldsymbol{p}, \boldsymbol{v}, R, \boldsymbol{\omega})$ is stored with 18 numbers $(3+3+9+3)$ but can only move in 12 independent directions, because R has just 3 degrees of freedom. Two operations are needed again and again, and they are the only place where R needs special care.

1. How far is the true state $\boldsymbol{x}$ from our estimate $\hat{\boldsymbol{x}}$? We describe it by the 12-number **error vector**

$$\delta\boldsymbol{x} = \begin{bmatrix} \delta\boldsymbol{p} \\ \delta\boldsymbol{v} \\ \delta\boldsymbol{\theta} \\ \delta\boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} \boldsymbol{p} - \hat{\boldsymbol{p}} \\ \boldsymbol{v} - \hat{\boldsymbol{v}} \\ \operatorname{Log}(\hat{R}^\top R) \\ \boldsymbol{\omega} - \hat{\boldsymbol{\omega}} \end{bmatrix} \in \mathbb{R}^{12}. \tag{6}$$

Position, velocity and rate errors are ordinary differences. The attitude error is the small rotation that turns $\hat{R}$ into R ($\hat{R}^\top R$ is "undo the estimate, then apply the truth"), written as a 3-vector by Log. Subtracting the matrices, $R - \hat{R}$, would give 9 numbers that do not describe a rotation.

2. How do we correct an estimate by a small error $\delta\boldsymbol{x}$? We do the reverse of (6):

$$\hat{\boldsymbol{p}} \leftarrow \hat{\boldsymbol{p}} + \delta\boldsymbol{p}, \qquad \hat{\boldsymbol{v}} \leftarrow \hat{\boldsymbol{v}} + \delta\boldsymbol{v}, \qquad \hat{R} \leftarrow \hat{R}\,\mathrm{Exp}(\delta\boldsymbol{\theta}), \qquad \hat{\boldsymbol{\omega}} \leftarrow \hat{\boldsymbol{\omega}} + \delta\boldsymbol{\omega}. \tag{7}$$

Three parts are plain addition. Only the attitude is corrected by *multiplying* with a small rotation, so that $\hat{R}$ remains a proper rotation matrix. Whenever this document says "correct the estimate by $\delta\boldsymbol{x}$", it means exactly these four lines.

Toy example — correcting an attitude

Estimate $\hat{R} = I$ (level, facing east). The filter decides the drone is actually yawed by $\delta\boldsymbol{\theta} = [0, 0, 0.1]\,\mathrm{rad}$ (5.7°):

$$\hat{R} \leftarrow I \cdot \mathrm{Exp}([0,0,0.1]) = \begin{bmatrix} 0.9950 & -0.0998 & 0 \\ 0.0998 & 0.9950 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

a pure 5.7° turn about the vertical axis. Naive addition $I + \delta$ could never produce the 0.9950 on the diagonal, and repeated additions would slowly turn $\hat{R}$ into a matrix that is no longer a rotation.

2.5 What was corrected with respect to the original images

Rederiving every equation exposed a few slips in the two hand-drawn flow images. The new flow diagrams (section 3) and this document use the corrected forms. The toy model demonstrates the most important one numerically.

#	Image / block	Issue and correction
1	1 / block 7	$\boldsymbol{e}_\omega = \boldsymbol{\omega}_d - \hat{\boldsymbol{\omega}}$ combined with $-K_\omega \boldsymbol{e}_\omega$ is *positive* rate feedback (it pumps energy in). Correct: $\boldsymbol{e}_\omega = \hat{\boldsymbol{\omega}} - \hat{R}^\top R_d \boldsymbol{\omega}_d$, as in `so3_controller.py`. In the toy model the image's sign makes the drone tumble within 0.07 s (fig. 4).
2	1 / block 4	Estimated state printed as $[\hat{\boldsymbol{p}}, \ddot{\boldsymbol{v}}, \hat{R}, \hat{\boldsymbol{\omega}}]$; it is $[\hat{\boldsymbol{p}}, \hat{\boldsymbol{v}}, \hat{R}, \hat{\boldsymbol{\omega}}]$.
3	1 / block 4	$P_{k+1} = AP_kA^\top + Q$ mixes the *continuous* Jacobian A with a discrete step. Use $F = I + A\Delta t \approx e^{A\Delta t}$: $P_{k+1} = FP_kF^\top + Q$.
4	1 / blocks 2–4	$\boldsymbol{x} - \hat{\boldsymbol{x}}$ and $\hat{\boldsymbol{x}} + K(\cdot)$ do not make sense for the rotation matrix R. Use the 12-number error (6) and the correction rule (7); this is what makes the filter an *error-state* EKF.
5	1 / block 7	$+J\dot{\boldsymbol{\omega}}_d$ is the small-error approximation of the full feed-forward $-J([\hat{\boldsymbol{\omega}}]_\times \hat{R}^\top R_d \boldsymbol{\omega}_d - \hat{R}^\top R_d \dot{\boldsymbol{\omega}}_d)$; both are given.
6	2 / blocks 1–2	$\boldsymbol{x}_k = [\boldsymbol{p}, \boldsymbol{v}, R, \boldsymbol{\omega}]_k$ must be a plain vector for DMDc: use $\mathrm{Log}(R_k)$ and deviations from hover, $\boldsymbol{u}_k = [T_k - mg_0; \boldsymbol{\tau}_k]$. Otherwise gravity adds a constant term that $x_{k+1} = Ax_k + Bu_k$ cannot represent.
7	2 / block 5	The index of the Riccati recursion counts *iterations* (steps-to-go), not time.

3 The Two Equation Flows

The project uses two complementary mathematical pipelines. Both end in the same four motor speeds.

- **Model 1 – model-based** (fig. 1). Newton's and Euler's laws give a nonlinear model. The model is linearized only where a linear model is really needed: inside the EKF, to propagate uncertainty. Estimation (VIO) and geometric control on SO(3) then run in a closed loop. This is the structure of our ROS 2 node `drones_controller`.
- **Model 2 – data-driven** (fig. 2). A linear model is *learned* from logged flight data (DMDc). An optimal controller is then designed on it: LQR (infinite horizon, no limits) or MPC (finite

horizon, explicit limits such as `max_velocity`). An extended Kalman filter on the same learned model (block 11) estimates the state from camera pixels and the gyroscope and feeds it back to the controllers.

How to read the diagrams. Each block has a coloured title bar (number, name, functional layer), the equations, and a footer that lists exactly what goes **IN** and what comes **OUT**, with dimensions. Arrow labels name the signal that travels between blocks. Block numbers equal the subsection numbers of sections 4 and 5 and the step numbers of the MATLAB toy model.

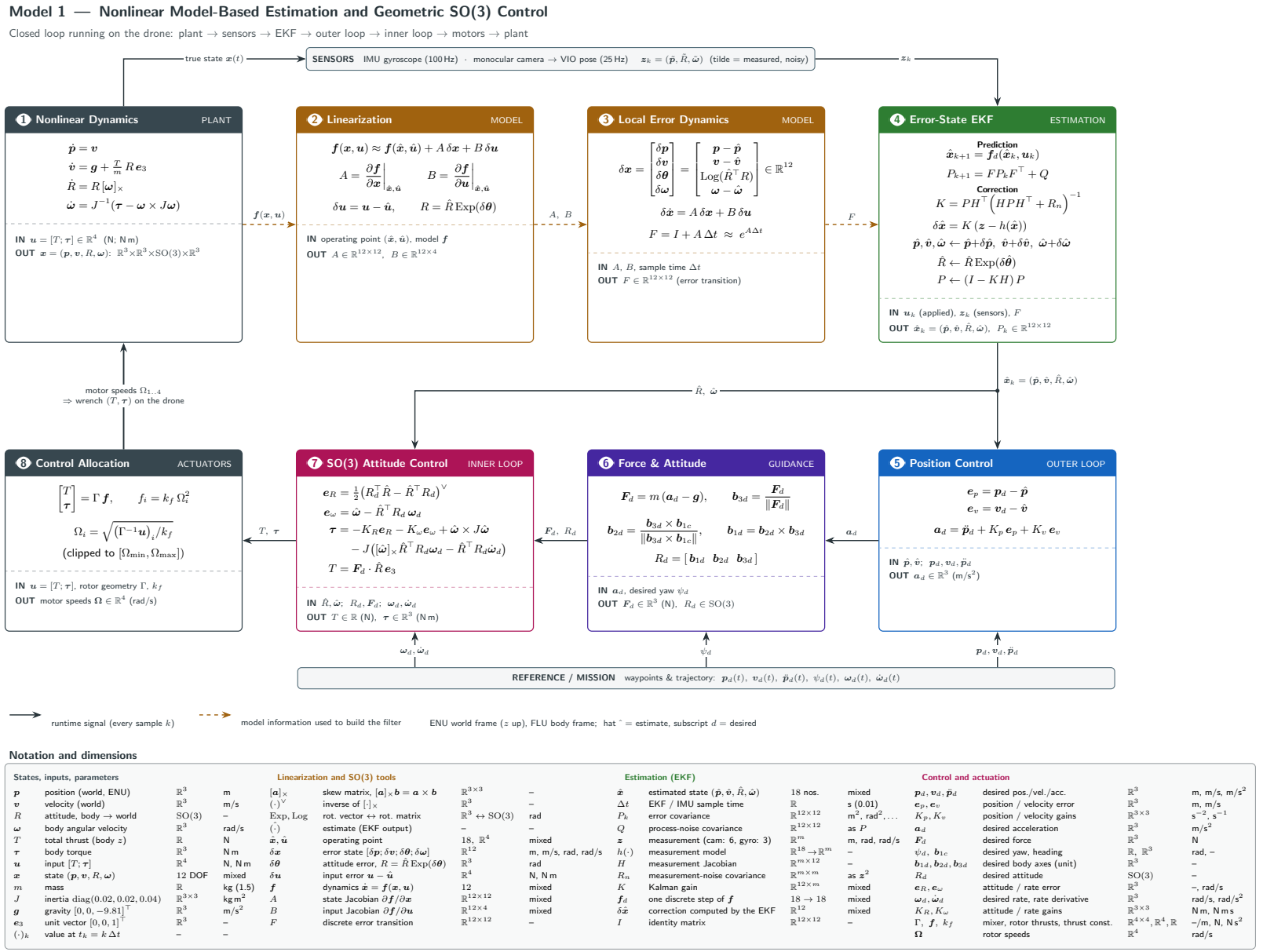


Figure 1: Model 1 equation flow: nonlinear model-based estimation and geometric SO(3) control. The top row models and estimates the state, the bottom row controls it, and the left edge closes the loop through the motors. Dashed arrows carry model information (Jacobians) and solid arrows carry run-time signals.

4 Model 1 — Nonlinear Model-Based Estimation and Geometric Control

Model 1 is the loop that actually flies the drone. It starts from physics (block 1). It builds the small-error linear model that an estimator needs (blocks 2–3) and estimates the state from the sensors (block 4). Finally it computes the commands in two nested loops, position (blocks 5–6) and attitude (block 7), and turns them into motor speeds (block 8).

4.1 Block 1 — Physical system and nonlinear dynamics

The drone is modelled as a single rigid body with mass m and inertia J, pushed by a thrust T along its own z-axis and twisted by a torque $\boldsymbol{\tau}$. Its state is position, velocity, attitude and angular velocity.

Equations M1.1 – M1.4 Rigid-body dynamics of the quadrotor

$$\dot{\boldsymbol{p}} = \boldsymbol{v} \tag{M1.1}$$

$$\dot{\boldsymbol{v}} = \boldsymbol{g} + \frac{T}{m} R\,\boldsymbol{e}_3 \tag{M1.2}$$

$$\dot{R} = R\,[\boldsymbol{\omega}]_\times \tag{M1.3}$$

$$\dot{\boldsymbol{\omega}} = J^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times J\boldsymbol{\omega}) \tag{M1.4}$$

Compactly $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ with state $\boldsymbol{x} = (\boldsymbol{p}, \boldsymbol{v}, R, \boldsymbol{\omega})$ and input $\boldsymbol{u} = [T; \boldsymbol{\tau}]$.

Symbol	Meaning	Dimension	Unit
$\boldsymbol{p}$, $\dot{\boldsymbol{p}}$	position and its rate (world frame)	$\mathbb{R}^3$	m, m/s
$\boldsymbol{v}$, $\dot{\boldsymbol{v}}$	velocity and acceleration (world frame)	$\mathbb{R}^3$	m/s, m/s^2
R, $\dot{R}$	attitude and its rate	$\mathbb{R}^{3\times3}$	–, 1/s
$\boldsymbol{\omega}$, $\dot{\boldsymbol{\omega}}$	body angular velocity and acceleration	$\mathbb{R}^3$	rad/s, rad/s^2
T	total thrust	$\mathbb{R}$	N
$\boldsymbol{\tau}$	body torque	$\mathbb{R}^3$	N m
$\boldsymbol{e}_3$	unit vector $[0,0,1]^\top$	$\mathbb{R}^3$	–
$\boldsymbol{x}$	state (stored as 18 numbers, 12 degrees of freedom)	$\mathbb{R}^3\times\mathbb{R}^3\times\mathrm{SO}(3)\times\mathbb{R}^3$	mixed
$\boldsymbol{u}$	input	$\mathbb{R}^4$	N, N m

Derivation

(M1.1) Kinematics. Velocity is by definition the time derivative of position: $\boldsymbol{v} = \frac{d\boldsymbol{p}}{dt}$.

(M1.2) Newton's second law. In the inertial (world) frame, $m\dot{\boldsymbol{v}} = \sum \boldsymbol{F}$. Two forces act:

- gravity, $m\boldsymbol{g}$;
- rotor thrust. In the body frame it is $T\boldsymbol{e}_3$ (straight "up" out of the propeller discs). Rotating it into the world frame gives $RT\boldsymbol{e}_3 = T\boldsymbol{b}_3$.

Hence $m\dot{\boldsymbol{v}} = m\boldsymbol{g} + TR\boldsymbol{e}_3$. Dividing by m gives (M1.2). Air drag and wind are neglected in the model. In the toy simulation they enter as an unknown disturbance, which is exactly what the controller must reject.

(M1.3) Attitude kinematics. Differentiate the constraint $R^\top R = I$:

$$\dot{R}^\top R + R^\top \dot{R} = 0 \quad\Longrightarrow\quad (R^\top \dot{R})^\top = -R^\top \dot{R}.$$

So $R^\top \dot{R}$ is skew-symmetric, which means it is the hat of some vector: $R^\top \dot{R} = [\boldsymbol{\omega}]_\times$, i.e. $\dot{R} = R[\boldsymbol{\omega}]_\times$. Physically, a point fixed on the body at $\boldsymbol{r}^{\mathcal{B}}$ sits at $R\boldsymbol{r}^{\mathcal{B}}$ in the world. Its velocity

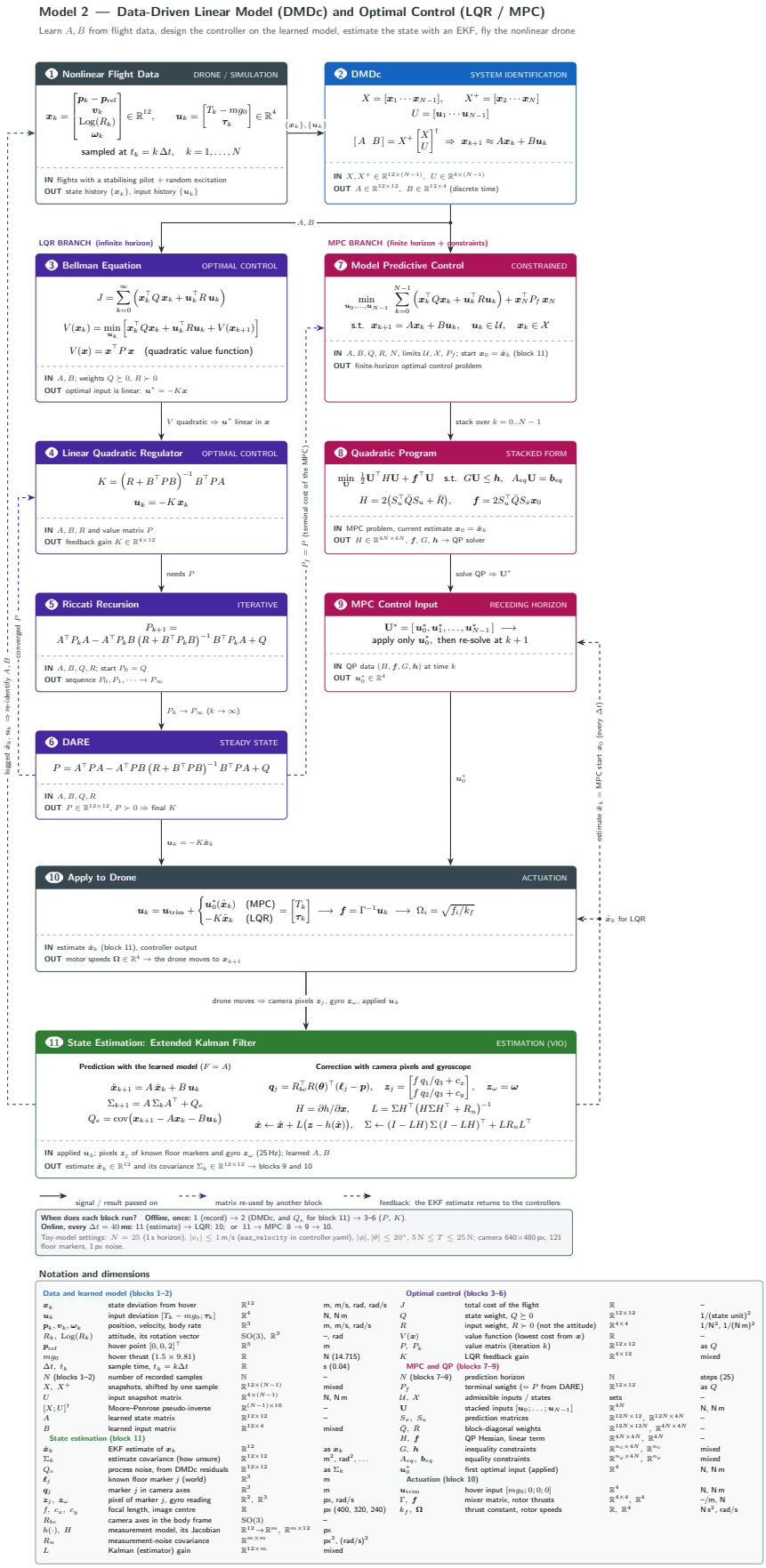


Figure 2: Model 2 equation flow: data-driven linear model (DMDc) followed by two optimal-control branches, LQR (left) and MPC (right), that meet when the command is applied to the drone. Block 11, an extended Kalman filter running on the learned model, turns camera pixels and gyro readings into the state estimate that both controllers use.

$\dot{R}\boldsymbol{r}^{\mathcal{B}} = R(\boldsymbol{\omega} \times \boldsymbol{r}^{\mathcal{B}})$ is a rotation about the axis $\boldsymbol{\omega}$, measured in the body frame. That is exactly what a gyroscope bolted to the drone reports.

(M1.4) Euler's rotation equation. The angular momentum is $J\boldsymbol{\omega}$ in the body frame and $\boldsymbol{h} = RJ\boldsymbol{\omega}$ in the world frame. Newton's law for rotation, $\dot{\boldsymbol{h}} = R\boldsymbol{\tau}$ (torque in world coordinates), together with (M1.3), gives

$$\frac{d}{dt}(RJ\boldsymbol{\omega}) = \dot{R}J\boldsymbol{\omega} + RJ\dot{\boldsymbol{\omega}} = R([\boldsymbol{\omega}]_\times J\boldsymbol{\omega} + J\dot{\boldsymbol{\omega}}) = R\boldsymbol{\tau} \implies J\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times J\boldsymbol{\omega} = \boldsymbol{\tau}.$$

The term $\boldsymbol{\omega} \times J\boldsymbol{\omega}$ is the *gyroscopic* coupling. A body spinning about two axes at once produces a rotation about the third axis without any torque.

Toy example — hover, tilt and gyroscopic coupling ($m = 1.5$, $J = \mathrm{diag}(0.02, 0.02, 0.04)$)

Hover. $R = I$, $\boldsymbol{\omega} = 0$, $T = mg_0 = 14.715\,\mathrm{N}$, $\boldsymbol{\tau} = 0$: $\dot{\boldsymbol{v}} = [0, 0, -9.81] + \frac{14.715}{1.5}[0, 0, 1] = \mathbf{0}$. The drone hangs still.
10° roll with hover thrust. $Re_3 = [0, -\sin 10°, \cos 10°]^\top$, so $\dot{\boldsymbol{v}} = g_0[0, -\sin 10°, \cos 10° - 1]^\top = [0, -1.7035, -0.1490]^\top\,\mathrm{m/s^2}$. It slides sideways and sinks slightly, because only $\cos 10° = 98.5\%$ of the thrust still holds it up.
Yaw rate. $R = I$, $\boldsymbol{\omega} = [0, 0, 0.5]^\top$: $\dot{R} = [\boldsymbol{\omega}]_\times = \begin{bmatrix} 0 & -0.5 & 0 \\ 0.5 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$, so the body x-axis starts to turn towards $+y$ at $0.5\,\mathrm{rad/s}$.
Gyroscopic coupling. $\boldsymbol{\omega} = [1, 0, 1]^\top$, $\boldsymbol{\tau} = 0$: $J\boldsymbol{\omega} = [0.02, 0, 0.04]^\top$, $\boldsymbol{\omega} \times J\boldsymbol{\omega} = [0, -0.02, 0]^\top$, so $\dot{\boldsymbol{\omega}} = J^{-1}[0, 0.02, 0]^\top = [0, 1, 0]^\top\,\mathrm{rad/s^2}$.

In our drone project

These four lines are "the real drone". In our set-up they are solved by Gazebo for the Iris quadrotor, while ArduPilot SITL plays the flight computer. Our ROS 2 node never integrates them. It *reads* their result on `/ap/v1/pose/filtered` (position $\boldsymbol{p}$ and orientation quaternion $\to R$) and `/ap/v1/twist/filtered` ($\boldsymbol{v}$, $\boldsymbol{\omega}$). Everything else in this document either predicts this motion (the EKF), steers it (the controller), or learns it from data (Model 2).

Toy model: `model1_nonlinear_ekf_so3/step1_nonlinear_dynamics.m`, `lib/quad_dynamics.m` (RK4 integration in `lib/rk4_step.m`). A $0.01\,\mathrm{N\,m}$ roll-torque pulse of $0.1\,\mathrm{s}$ already makes the open-loop drone drift $2.1\,\mathrm{m}$ in $4\,\mathrm{s}$ (`figures/step1_open_loop.png`).

4.2 Block 2 — Linearization: Taylor expansion and Jacobians

Equations M1.5 – M1.6 First-order Taylor expansion and Jacobians

$$\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) \approx \boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}) + A\,\delta\boldsymbol{x} + B\,\delta\boldsymbol{u}, \qquad \delta\boldsymbol{x} \text{ from (6)}, \quad \delta\boldsymbol{u} = \boldsymbol{u} - \hat{\boldsymbol{u}} \tag{M1.5}$$

$$A = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}} \in \mathbb{R}^{12\times 12}, \qquad B = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\right|_{\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}} \in \mathbb{R}^{12\times 4} \tag{M1.6}$$

Symbol	Meaning	Dimension	Unit
$\hat{\boldsymbol{x}}$, $\hat{\boldsymbol{u}}$	operating point (where we linearize)	$\mathbb{R}^{18}$, $\mathbb{R}^4$	mixed
$\delta\boldsymbol{x}$	state error $[\boldsymbol{p} - \hat{\boldsymbol{p}};\ \boldsymbol{v} - \hat{\boldsymbol{v}};\ \mathrm{Log}(\hat{R}^\top R);\ \boldsymbol{\omega} - \hat{\boldsymbol{\omega}}]$	$\mathbb{R}^{12}$	m, m/s, rad, rad/s
$\delta\boldsymbol{u} = \boldsymbol{u} - \hat{\boldsymbol{u}}$	input error $[\delta T; \delta\boldsymbol{\tau}]$	$\mathbb{R}^4$	N, N m
A	state Jacobian	$\mathbb{R}^{12\times 12}$	mixed (1/s, m/s²…)
B	input Jacobian	$\mathbb{R}^{12\times 4}$	mixed (1/kg, 1/(kg m²))

Derivation of the Taylor expansion

For a smooth vector function, Taylor's theorem in several variables states

$$\boldsymbol{f}(\hat{\boldsymbol{x}}+\delta\boldsymbol{x},\hat{\boldsymbol{u}}+\delta\boldsymbol{u}) = \boldsymbol{f}(\hat{\boldsymbol{x}},\hat{\boldsymbol{u}}) + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\delta\boldsymbol{x} + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\delta\boldsymbol{u} + \mathcal{O}(\|\delta\boldsymbol{x}\|^2 + \|\delta\boldsymbol{u}\|^2).$$

Dropping the second-order remainder gives (M1.5). Entry (i,j) of A is $\partial f_i/\partial x_j$, i.e. how much the rate of state i changes when state j is nudged. Because R lives on SO(3), "nudging" the attitude means $R = \hat{R}\,\mathrm{Exp}(\delta\boldsymbol{\theta})$, so the attitude part of $\delta\boldsymbol{x}$ has 3 entries, not 9.

Derivation of A and B for the drone

Insert $\boldsymbol{p} = \hat{\boldsymbol{p}} + \delta\boldsymbol{p}$, $\boldsymbol{v} = \hat{\boldsymbol{v}} + \delta\boldsymbol{v}$, $R = \hat{R}\mathrm{Exp}(\delta\boldsymbol{\theta}) \approx \hat{R}(I + [\delta\boldsymbol{\theta}]_\times)$, $\boldsymbol{\omega} = \hat{\boldsymbol{\omega}} + \delta\boldsymbol{\omega}$, $T = \hat{T} + \delta T$, $\boldsymbol{\tau} = \hat{\boldsymbol{\tau}} + \delta\boldsymbol{\tau}$ into (M1.1)–(M1.4) and keep first-order terms.

- **Position:** $\delta\dot{\boldsymbol{p}} = \delta\boldsymbol{v}$.
- **Velocity:** $\dot{\boldsymbol{v}} \approx \boldsymbol{g} + \frac{\hat{T}+\delta T}{m}\hat{R}(I + [\delta\boldsymbol{\theta}]_\times)\boldsymbol{e}_3$. Using $[\delta\boldsymbol{\theta}]_\times\boldsymbol{e}_3 = \delta\boldsymbol{\theta}\times\boldsymbol{e}_3 = -[\boldsymbol{e}_3]_\times\delta\boldsymbol{\theta}$ and dropping the product $\delta T\,\delta\boldsymbol{\theta}$:
$$\delta\dot{\boldsymbol{v}} = -\frac{\hat{T}}{m}\hat{R}[\boldsymbol{e}_3]_\times\,\delta\boldsymbol{\theta} + \frac{1}{m}\hat{R}\boldsymbol{e}_3\,\delta T.$$
- **Attitude:** differentiate $R = \hat{R}\mathrm{Exp}(\delta\boldsymbol{\theta})$ with $\dot{R} = R[\boldsymbol{\omega}]_\times$ and $\dot{\hat{R}} = \hat{R}[\hat{\boldsymbol{\omega}}]_\times$. To first order, $[\delta\dot{\boldsymbol{\theta}}]_\times = [\delta\boldsymbol{\omega}]_\times + [\delta\boldsymbol{\theta}]_\times[\hat{\boldsymbol{\omega}}]_\times - [\hat{\boldsymbol{\omega}}]_\times[\delta\boldsymbol{\theta}]_\times = [\delta\boldsymbol{\omega}]_\times - [\hat{\boldsymbol{\omega}}\times\delta\boldsymbol{\theta}]_\times$, so
$$\delta\dot{\boldsymbol{\theta}} = -[\hat{\boldsymbol{\omega}}]_\times\,\delta\boldsymbol{\theta} + \delta\boldsymbol{\omega}.$$
- **Angular rate:** $\frac{\partial}{\partial\boldsymbol{\omega}}(\boldsymbol{\omega}\times J\boldsymbol{\omega}) = [\boldsymbol{\omega}]_\times J - [J\boldsymbol{\omega}]_\times$, hence
$$\delta\dot{\boldsymbol{\omega}} = J^{-1}\big([J\hat{\boldsymbol{\omega}}]_\times - [\hat{\boldsymbol{\omega}}]_\times J\big)\delta\boldsymbol{\omega} + J^{-1}\delta\boldsymbol{\tau}.$$

Stacking the four rows gives

$$A = \begin{bmatrix} 0 & I & 0 & 0 \\ 0 & 0 & -\frac{\hat{T}}{m}\hat{R}[\boldsymbol{e}_3]_\times & 0 \\ 0 & 0 & -[\hat{\boldsymbol{\omega}}]_\times & I \\ 0 & 0 & 0 & J^{-1}\big([J\hat{\boldsymbol{\omega}}]_\times - [\hat{\boldsymbol{\omega}}]_\times J\big) \end{bmatrix}, \qquad B = \begin{bmatrix} 0 & 0 \\ \frac{1}{m}\hat{R}\boldsymbol{e}_3 & 0 \\ 0 & 0 \\ 0 & J^{-1} \end{bmatrix}. \tag{8}$$

At hover ($\hat{R} = I$, $\hat{\boldsymbol{\omega}} = 0$, $\hat{T} = mg_0$) the key blocks become

$$-g_0[\boldsymbol{e}_3]_\times = \begin{bmatrix} 0 & 9.81 & 0 \\ -9.81 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad \tfrac{1}{m}\boldsymbol{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 0.667 \end{bmatrix}, \qquad J^{-1} = \mathrm{diag}(50, 50, 25).$$

Read row by row: a pitch error $\delta\theta_y$ accelerates the drone along $+x$ at $9.81\,\delta\theta_y$ m/s^2. A roll error $\delta\theta_x$ accelerates it along $-y$. Extra thrust only changes vertical acceleration. Torque changes angular acceleration with gain $1/J$.

Toy example — how good is "first order"?

Pitch the hovering drone by θ. The linear model predicts $\dot{v}_x = g_0\theta$; the truth is $g_0\sin\theta$.

θ [deg]	linear $g_0\theta$ [m/s^2]	true $g_0 \sin\theta$ [m/s^2]	error
1.00	0.1712	0.1712	0.005%
2.87 (0.05 rad)	0.4905	0.4903	0.04%
10.00	1.7122	1.7035	0.51%
20.00	3.4243	3.3552	2.06%
28.65 (0.5 rad)	4.9054	4.7035	4.29%

The analytic Jacobians (8) were also checked against numerical differentiation of the true flow at a tilted, spinning, accelerating state. The largest difference is 2.3×10^{-8} for A and 8.4×10^{-7} for B.

In our drone project

Linearization is the "zoom-in" trick. Near any flight condition the complicated drone behaves like a simple linear system, and linear systems are easy to reason about. Our EKF uses A to answer one question every 10 ms: *how fast does my uncertainty grow if I trust only the physics?* The table shows the trick is excellent for the gentle tilts of normal flight (below 10°, under 0.5% error). That is why both the EKF and Model 2 can rely on it.

Toy model: `step2_taylor_jacobian.m`, `lib/quad_jacobians.m`; figure `step2_taylor_accuracy.png`.

4.3 Block 3 — Local error dynamics

Equations M1.7 – M1.9 Linear dynamics of the estimation error

$$\delta\boldsymbol{x} = [\boldsymbol{p} - \hat{\boldsymbol{p}};\ \boldsymbol{v} - \hat{\boldsymbol{v}};\ \mathrm{Log}(\hat{R}^\top R);\ \boldsymbol{\omega} - \hat{\boldsymbol{\omega}}] \tag{M1.7}$$

$$\delta\dot{\boldsymbol{x}} = A\,\delta\boldsymbol{x} + B\,\delta\boldsymbol{u} \tag{M1.8}$$

$$\delta\boldsymbol{x}_{k+1} = F\,\delta\boldsymbol{x}_k + G\,\delta\boldsymbol{u}_k, \qquad F = e^{A\Delta t} \approx I + A\Delta t, \quad G \approx B\Delta t \tag{M1.9}$$

Symbol	Meaning	Dimension	Unit
$\delta\boldsymbol{x}$	error between true and estimated (or nominal) state	$\mathbb{R}^{12}$	mixed
F	discrete error-transition matrix	$\mathbb{R}^{12\times12}$	–
G	discrete input matrix	$\mathbb{R}^{12\times4}$	mixed
Δt	sample time	$\mathbb{R}$	s (0.01)

Derivation

The true state obeys $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ and the reference (estimated) state obeys the same model, $\dot{\hat{\boldsymbol{x}}} = \boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}})$. Subtracting, and using (M1.5),

$$\delta\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) - \boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}) \approx \big[\boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}) + A\delta\boldsymbol{x} + B\delta\boldsymbol{u}\big] - \boldsymbol{f}(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}) = A\delta\boldsymbol{x} + B\delta\boldsymbol{u}.$$

The large nonlinear parts cancel and only the small, linear error remains. Over one sample, with A and $\delta\boldsymbol{u}$ held constant, the solution of this linear ODE is $\delta\boldsymbol{x}(t_k + \Delta t) = e^{A\Delta t}\delta\boldsymbol{x}(t_k) + \int_0^{\Delta t} e^{As}\,ds\,B\,\delta\boldsymbol{u}_k$. The series $e^{A\Delta t} = I + A\Delta t + \frac{1}{2}(A\Delta t)^2 + \ldots$ truncated after the linear term gives (M1.9). This is exactly the forward-Euler step.

Why the error grows without feedback. At hover every eigenvalue of A is zero (A is nilpotent). The chain $\delta\boldsymbol{\omega} \to \delta\boldsymbol{\theta} \to \delta\boldsymbol{v} \to \delta\boldsymbol{p}$ is four integrators in series, so a constant rate error grows into a position error proportional to t^3. The drone is *open-loop unstable*. That is why it needs both a good estimate (block 4) and feedback (blocks 5–7).

Toy example — propagate an error for 2 s

Extra thrust $\delta T = 0.5\,\mathrm{N}$ at hover. Only row 6 of B is non-zero, so $\delta v_z(t) = \frac{\delta T}{m} t = \frac{0.5}{1.5} \cdot 2 = 0.667\,\mathrm{m/s}$. The linear model gives 0.6667 and the nonlinear truth 0.6543.
Initial error 2° roll, 10 cm, 0.2 rad/s yaw rate: after 2 s the linear prediction differs from the truth by 2.6%. With a 30° initial roll the difference is 26%. The linear model is *local*.
With $\Delta t = 0.01\,\mathrm{s}$, $\max |F - e^{A\Delta t}| = 4.9 \times 10^{-4}$, the size of the neglected $\frac{1}{2}(A\Delta t)^2$ term.

In our drone project

The EKF never tries to describe the whole flight with a linear model. It only describes the *small difference* between what it believes and what is true, and that difference *is* small, because the camera and IMU keep correcting it. This is the reason for the name *error-state* filter.

Toy model: `step3_local_error_dynamics.m`; figure `step3_error_dynamics.png`.

4.4 Block 4 — State estimation: error-state EKF

The idea before the maths. The filter repeats two steps forever.

1. **Predict.** "I know how I pushed the drone, so the physics tells me where it should be now." The guess moves forward, and because physics is never perfect, the uncertainty *grows*.
2. **Correct.** "A sensor reading has arrived. How surprised am I?" The guess is pulled towards the measurement, by an amount that depends on whom we trust more, and the uncertainty *shrinks*.

The filter stores two things: the **estimate** $\hat{\boldsymbol{x}}_k = (\hat{\boldsymbol{p}}, \hat{\boldsymbol{v}}, \hat{R}, \hat{\boldsymbol{\omega}})$ (its best guess) and the **covariance** P_k (how unsure it is about each of the 12 error directions of (6)).

Assumptions about noise. The real drone follows the model only approximately. Each step it is pushed by a small unknown random error $\boldsymbol{w}_k$ (wind, model mistakes) with covariance Q. Every sensor reads the true value plus a random error $\boldsymbol{v}_k$ with covariance R_n:

$$\boldsymbol{x}_{k+1} = \boldsymbol{f}_d(\boldsymbol{x}_k, \boldsymbol{u}_k) \text{ plus a random error } \boldsymbol{w}_k \sim \mathcal{N}(0, Q), \qquad \boldsymbol{z}_k = h(\boldsymbol{x}_k) + \boldsymbol{v}_k, \quad \boldsymbol{v}_k \sim \mathcal{N}(0, R_n).$$

Here $\boldsymbol{f}_d$ is one simulation step (RK4) of (M1.1)–(M1.4), and $\mathcal{N}(0, Q)$ means "Gaussian with mean zero and covariance Q". The errors are added in the 12 error directions, so for the attitude "adding" means a small rotation, as in (7).

Equations M1.10 – M1.14 Error-state EKF

Time update (prediction)

$$\hat{\boldsymbol{x}}_{k+1} = \boldsymbol{f}_d(\hat{\boldsymbol{x}}_k, \boldsymbol{u}_k) \tag{M1.10}$$
$$P_{k+1} = F P_k F^\top + Q \tag{M1.11}$$

Measurement update (correction)

$$K = P H^\top (H P H^\top + R_n)^{-1} \tag{M1.12}$$
$$\delta\hat{\boldsymbol{x}} = K\,\boldsymbol{r}, \qquad \boldsymbol{r} = \boldsymbol{z} - h(\hat{\boldsymbol{x}}) \ \text{(innovation)}, \qquad \text{then correct } \hat{\boldsymbol{x}} \text{ by } \delta\hat{\boldsymbol{x}} \text{ using (7)} \tag{M1.13}$$
$$P \leftarrow (I - KH)\,P \tag{M1.14}$$

Written out, the correction in (M1.13) is $\hat{\boldsymbol{p}} \leftarrow \hat{\boldsymbol{p}} + \delta\hat{\boldsymbol{p}}$, $\hat{\boldsymbol{v}} \leftarrow \hat{\boldsymbol{v}} + \delta\hat{\boldsymbol{v}}$, $\hat{R} \leftarrow \hat{R}\,\mathrm{Exp}(\delta\hat{\boldsymbol{\theta}})$, $\hat{\boldsymbol{\omega}} \leftarrow \hat{\boldsymbol{\omega}} + \delta\hat{\boldsymbol{\omega}}$.

Symbol	Meaning	Dimension	Unit
$\hat{\boldsymbol{x}}_k$	estimated state $(\hat{\boldsymbol{p}}, \hat{\boldsymbol{v}}, \hat{R}, \hat{\boldsymbol{\omega}})$	$\mathbb{R}^{18}$	mixed
P_k	error covariance	$\mathbb{R}^{12\times12}$	m^2, $(\mathrm{m/s})^2$, rad^2, …
Q	process-noise covariance per step (wind, model error)	$\mathbb{R}^{12\times12}$	as P
$\boldsymbol{z}$	measurement (camera pose: $m=6$, gyro: $m=3$)	$\mathbb{R}^m$	m, rad, rad/s
$h(\cdot)$	measurement model	$\mathbb{R}^{18}\to\mathbb{R}^m$	–
$H=\partial h/\partial\delta\boldsymbol{x}$	measurement Jacobian	$\mathbb{R}^{m\times12}$	–
R_n	measurement-noise covariance	$\mathbb{R}^{m\times m}$	m^2, rad^2, …
K	Kalman gain	$\mathbb{R}^{12\times m}$	mixed
$\boldsymbol{r}=\boldsymbol{z}-h(\hat{\boldsymbol{x}})$	innovation: measured minus expected (“surprise”)	$\mathbb{R}^m$	as $\boldsymbol{z}$
$\delta\hat{\boldsymbol{x}}$	correction computed from the innovation	$\mathbb{R}^{12}$	mixed

Derivation

(M1.10) Mean prediction. The truth is the estimate corrected by an unknown error $\delta\boldsymbol{x}_k$ whose average is zero (a good estimate is not biased to one side). By the linearization of block 3, one step later the truth is $\boldsymbol{f}_d(\hat{\boldsymbol{x}}_k,\boldsymbol{u}_k)$ corrected by $F\delta\boldsymbol{x}_k$. Since $\delta\boldsymbol{x}_k$ averages to zero, so does $F\delta\boldsymbol{x}_k$, and the best guess of the next state is simply $\boldsymbol{f}_d(\hat{\boldsymbol{x}}_k,\boldsymbol{u}_k)$. The full nonlinear model is used for the guess itself. The linear model is only needed for the uncertainty.

(M1.11) Covariance prediction. From (M1.9) plus noise, $\delta\boldsymbol{x}_{k+1}=F\delta\boldsymbol{x}_k+\boldsymbol{w}_k$. Then

$$P_{k+1}=\mathbb{E}[\delta\boldsymbol{x}_{k+1}\delta\boldsymbol{x}_{k+1}^\top]=F\,\mathbb{E}[\delta\boldsymbol{x}_k\delta\boldsymbol{x}_k^\top]F^\top+\mathbb{E}[\boldsymbol{w}_k\boldsymbol{w}_k^\top]=FP_kF^\top+Q,$$

because the noise is independent of the current error, so the cross terms vanish. Uncertainty is stretched by the dynamics ($F\cdot F^\top$) and inflated by the noise ($+Q$).

(M1.12) Kalman gain. Linearize the measurement: the innovation $\boldsymbol{r}=\boldsymbol{z}-h(\hat{\boldsymbol{x}})\approx H\delta\boldsymbol{x}+\boldsymbol{v}$ (“what I see minus what I expected = my error seen through the sensor, plus sensor noise”). We correct the error estimate by a linear rule $\delta\hat{\boldsymbol{x}}=K\boldsymbol{r}$. The error after the correction is $\delta\boldsymbol{x}^+=\delta\boldsymbol{x}-K(H\delta\boldsymbol{x}+\boldsymbol{v})=(I-KH)\delta\boldsymbol{x}-K\boldsymbol{v}$, whose covariance is

$$P^+=(I-KH)P(I-KH)^\top+KR_nK^\top\qquad\text{(Joseph form, valid for any } K\text{).}\tag{9}$$

The best K minimises the total variance $\operatorname{tr}P^+$. Using $\frac{\partial}{\partial K}\operatorname{tr}(KMK^\top)=2KM$ for symmetric M and $\frac{\partial}{\partial K}\operatorname{tr}(KN)=N^\top$:

$$\frac{\partial\operatorname{tr}P^+}{\partial K}=-2PH^\top+2K\big(HPH^\top+R_n\big)=0\implies K=PH^\top\underbrace{\big(HPH^\top+R_n\big)}_{S\ \text{(innovation covariance)}}{}^{-1}.$$

(M1.13) State update. The best estimate of the error is $\delta\hat{\boldsymbol{x}}=K\boldsymbol{r}$. It is applied to the estimate with the correction rule (7): positions, velocities and rates are added, and the attitude is rotated, $\hat{R}\leftarrow\hat{R}\mathrm{Exp}(\delta\hat{\boldsymbol{\theta}})$, so $\hat{R}$ stays a valid rotation matrix. The error has now been absorbed into the estimate, so the filter starts the next step believing its error is zero on average again.

(M1.14) Covariance update. Expand (9) and use $KS=PH^\top$: $P^+=P-KHP-PH^\top K^\top+KSK^\top=P-KHP-PH^\top K^\top+PH^\top K^\top=(I-KH)P$. The toy code uses this short form and re-symmetrises P after every step.

Our measurement models

Sensor (rate)	innovation $\boldsymbol{r}$	H	noise $\sqrt{\operatorname{diag}R_n}$
Gyroscope (100 Hz)	$\tilde{\boldsymbol{\omega}}-\hat{\boldsymbol{\omega}}$	$\begin{bmatrix}0&0&0&I\end{bmatrix}$	0.01 rad/s
Camera / VIO pose (25 Hz)	$\begin{bmatrix}\tilde{\boldsymbol{p}}-\hat{\boldsymbol{p}}\\ \mathrm{Log}(\hat{R}^\top\tilde{R})\end{bmatrix}$	$\begin{bmatrix}I&0&0&0\\0&0&I&0\end{bmatrix}$	0.05 m, 0.02 rad

Each block of H is 3×3, and the columns follow $[\delta\boldsymbol{p}, \delta\boldsymbol{v}, \delta\boldsymbol{\theta}, \delta\boldsymbol{\omega}]$. Velocity is *never measured*. The filter infers it from how the measured position changes and from the physics in F.

Toy example — one predict + one correct, altitude only

Prior: altitude $\hat{z}_k = 10\,\text{m}$, variance $P_k = 3.5\,\text{m}^2$. Commanded climb $0.5\,\text{m/s}$ for $\Delta t = 1\,\text{s}$, $F = 1$, $Q = 0.5$:

$$\hat{z}^- = 10 + 0.5 = 10.5\ \text{m}, \qquad P^- = 1 \cdot 3.5 \cdot 1 + 0.5 = 4.0\ \text{m}^2.$$

GPS reads $z = 12.5\,\text{m}$ with $R_n = 1\,\text{m}^2$, $H = 1$:

$$K = \frac{4}{4+1} = 0.8, \quad \hat{z}^+ = 10.5 + 0.8\,(12.5 - 10.5) = 12.1\ \text{m}, \quad P^+ = (1 - 0.8) \cdot 4 = 0.8\ \text{m}^2.$$

The GPS is four times more trustworthy than the prediction ($P^-/R_n = 4$), so the estimate moves 80% of the way towards it, and the variance shrinks from 4.0 to 0.8.
Full 12-state filter (20 s helix flight, gusts, noisy sensors, wrong initial guess):

quantity	RMSE after 2 s	for comparison
position	2.55 cm	raw camera: 8.55 cm
velocity (never measured)	3.9 cm/s	–
attitude	0.20°	camera noise: 1.15°
body rate	0.0025 rad/s	gyro noise: 0.01 rad/s

In 99% of the samples all 12 errors lie inside the $\pm 3\sigma$ band predicted by P_k, so the filter is *consistent*: it knows how uncertain it is.

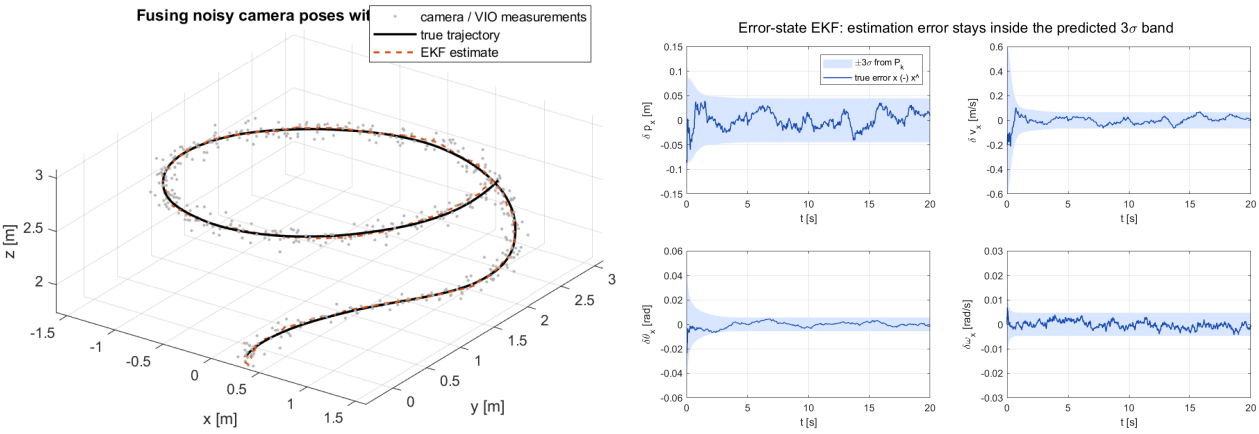


Figure 3: Toy model, Model 1 step 4. Left: noisy camera positions (grey), truth (black) and EKF estimate. Right: estimation errors stay inside the $\pm 3\sigma$ band computed from P_k.

In our drone project — this is the "VIO" of our title

A monocular camera alone cannot tell scale (a small room close up looks like a big room far away), and an IMU alone drifts within seconds. The EKF is the referee that fuses them. Between camera frames it predicts with the physics and the gyro. When a frame arrives, the visual-odometry pose corrects the prediction, and K decides how much to trust each source. Its output $\hat{\boldsymbol{x}}$ (position, velocity, attitude, rate) is the only thing the controller ever sees. In the current SITL set-up ArduPilot's own EKF3 produces `/ap/v1/*/filtered`. Our VIO pipeline supplies the camera measurement $\boldsymbol{z}_{\text{cam}}$ to exactly this structure.

Toy model: `step4_error_state_ekf.m`, `lib/ekf_predict.m`, `lib/ekf_update.m`.

4.5 Block 5 — Position control (outer loop)

Equations M1.15 – M1.16 Tracking errors and desired acceleration

$$\boldsymbol{e}_p = \boldsymbol{p}_d - \hat{\boldsymbol{p}}, \qquad \boldsymbol{e}_v = \boldsymbol{v}_d - \hat{\boldsymbol{v}} \tag{M1.15}$$

$$\boldsymbol{a}_d = \ddot{\boldsymbol{p}}_d + K_p\,\boldsymbol{e}_p + K_v\,\boldsymbol{e}_v \tag{M1.16}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{p}_d,\ \boldsymbol{v}_d,\ \ddot{\boldsymbol{p}}_d$	desired position, velocity, acceleration	$\mathbb{R}^3$	m, m/s, m/s^2
$\boldsymbol{e}_p,\ \boldsymbol{e}_v$	position and velocity error	$\mathbb{R}^3$	m, m/s
K_p	position gain (diagonal, positive)	$\mathbb{R}^{3\times3}$	1/s^2
K_v	velocity gain (diagonal, positive)	$\mathbb{R}^{3\times3}$	1/s
$\boldsymbol{a}_d$	desired acceleration	$\mathbb{R}^3$	m/s^2

Derivation

Suppose the inner loops achieve the commanded acceleration exactly, $\ddot{\boldsymbol{p}} = \boldsymbol{a}_d$. Then the position error obeys

$$\ddot{\boldsymbol{e}}_p = \ddot{\boldsymbol{p}}_d - \ddot{\boldsymbol{p}} = \ddot{\boldsymbol{p}}_d - \boldsymbol{a}_d = -K_p\boldsymbol{e}_p - K_v\dot{\boldsymbol{e}}_p \quad\Longrightarrow\quad \ddot{\boldsymbol{e}}_p + K_v\dot{\boldsymbol{e}}_p + K_p\boldsymbol{e}_p = \boldsymbol{0}.$$

With diagonal gains, each axis is a mass–spring–damper with characteristic polynomial $s^2 + k_v s + k_p$. Both roots have negative real part if and only if $k_p, k_v > 0$, so the error decays to zero. Matching with $s^2 + 2\zeta\omega_n s + \omega_n^2$:

$$\omega_n = \sqrt{k_p}, \qquad \zeta = \frac{k_v}{2\sqrt{k_p}}, \qquad t_{2\%} \approx \frac{4}{\zeta\omega_n}.$$

The feed-forward $\ddot{\boldsymbol{p}}_d$ supplies the acceleration a perfect pilot would need. The feedback terms only fix what is left over.

axis	k_p	k_v	ω_n [rad/s]	ζ	$t_{2\%}$ [s]
$x,\ y$	1.5	1.2	1.225	0.490	6.7
z	2.0	1.5	1.414	0.530	5.3

These are our `controller.yaml` gains: gentle, slightly under-damped, a little overshoot, settling in about 6 s.

Toy example — 50 cm low and climbing

Setpoint $\boldsymbol{p}_d = [0, 0, 2]$ m (the `controller.yaml` hover point), $\boldsymbol{v}_d = \ddot{\boldsymbol{p}}_d = \boldsymbol{0}$. Estimate $\hat{\boldsymbol{p}} = [0.1,\ -0.2,\ 1.5]$ m, $\hat{\boldsymbol{v}} = [0, 0, 0.3]$ m/s.

$$\boldsymbol{e}_p = [-0.1,\ 0.2,\ 0.5], \quad \boldsymbol{e}_v = [0,\ 0,\ -0.3], \quad \boldsymbol{a}_d = \begin{bmatrix} 1.5(-0.1) \\ 1.5(0.2) \\ 2(0.5) + 1.5(-0.3) \end{bmatrix} = \begin{bmatrix} -0.15 \\ 0.30 \\ 0.55 \end{bmatrix} \text{ m/s}^2.$$

It is 50 cm low, so it wants to accelerate up. It is already climbing at 0.3 m/s, so it asks for less (0.55 instead of 1.0).

In our drone project

This is the "where am I versus where should I be" layer: `SO3Controller.compute_position_errors()` and the first line of `compute_desired_force()` in `so3_controller.py`. The positions come from the EKF, and the setpoint comes from `desired_x/y/z` in the YAML (or a waypoint generator). Large K_p means an aggressive drone, and large K_v means a damped, smooth drone.

Toy model: `step5_position_control.m`, `lib/position_control.m`; figure `step5_error_decay.png`.

4.6 Block 6 — Force and attitude generation

Equations M1.17 – M1.19 Desired force, thrust direction and attitude

$$\boldsymbol{F}_d = m\,(\boldsymbol{a}_d - \boldsymbol{g}) \tag{M1.17}$$

$$\boldsymbol{b}_{3d} = \frac{\boldsymbol{F}_d}{\|\boldsymbol{F}_d\|} \tag{M1.18}$$

$$\boldsymbol{b}_{2d} = \frac{\boldsymbol{b}_{3d} \times \boldsymbol{b}_{1c}}{\|\boldsymbol{b}_{3d} \times \boldsymbol{b}_{1c}\|}, \quad \boldsymbol{b}_{1d} = \boldsymbol{b}_{2d} \times \boldsymbol{b}_{3d}, \quad R_d = [\boldsymbol{b}_{1d} \;\; \boldsymbol{b}_{2d} \;\; \boldsymbol{b}_{3d}], \quad \boldsymbol{b}_{1c} = [\cos\psi_d,\ \sin\psi_d,\ 0]^\top \tag{M1.19}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{F}_d$	desired total force vector (world)	$\mathbb{R}^3$	N
$\boldsymbol{b}_{1d}, \boldsymbol{b}_{2d}, \boldsymbol{b}_{3d}$	desired body axes (unit vectors)	$\mathbb{R}^3$	–
ψ_d	desired yaw (heading)	$\mathbb{R}$	rad
$\boldsymbol{b}_{1c}$	heading direction in the horizontal plane	$\mathbb{R}^3$	–
R_d	desired attitude	SO(3)	–

Derivation

(M1.17) We want $\dot{\boldsymbol{v}} = \boldsymbol{a}_d$. From (M1.2), $\boldsymbol{a}_d = \boldsymbol{g} + \frac{T}{m} R\boldsymbol{e}_3$, so the thrust vector must satisfy $T\,R\boldsymbol{e}_3 = m(\boldsymbol{a}_d - \boldsymbol{g}) = \boldsymbol{F}_d$. In ENU $-\boldsymbol{g} = [0, 0, +g_0]$, so even for $\boldsymbol{a}_d = 0$ the drone must push up with mg_0. In the code's notation, $\boldsymbol{F}_d = m(\boldsymbol{a}_{\text{cmd}} + g\boldsymbol{e}_3)$, which is the same expression.

(M1.18) Underactuation. A quadrotor has 4 inputs but 6 degrees of freedom. It cannot push sideways. The only way to produce $\boldsymbol{F}_d$ is to point the body axis $\boldsymbol{b}_3 = R\boldsymbol{e}_3$ along it. Because $\|R\boldsymbol{e}_3\| = 1$, matching $T\boldsymbol{b}_3 = \boldsymbol{F}_d$ forces $T = \|\boldsymbol{F}_d\|$ and $\boldsymbol{b}_{3d} = \boldsymbol{F}_d/\|\boldsymbol{F}_d\|$. Translation is controlled *through* the attitude.

(M1.19) Completing the rotation. $\boldsymbol{b}_{3d}$ fixes two of the three attitude angles. The remaining freedom is yaw, which we choose. By construction: $\boldsymbol{b}_{2d} \perp \boldsymbol{b}_{3d}$ and $\boldsymbol{b}_{2d} \perp \boldsymbol{b}_{1c}$ (cross product); $\boldsymbol{b}_{1d} = \boldsymbol{b}_{2d} \times \boldsymbol{b}_{3d}$ is perpendicular to both and has unit length, since it is the cross product of orthogonal unit vectors. The columns are therefore orthonormal, and since $\boldsymbol{b}_{1d} \times \boldsymbol{b}_{2d} = \boldsymbol{b}_{3d}$ the frame is right-handed. Hence $R_d^\top R_d = I$ and $\det R_d = +1$, so $R_d \in \mathrm{SO}(3)$. $\boldsymbol{b}_{1d}$ is the projection of the desired heading onto the plane perpendicular to the thrust. The construction fails only if $\boldsymbol{b}_{3d} \parallel \boldsymbol{b}_{1c}$ (a 90° tilt). The code then nudges ψ_d by 10^{-3} rad.

Toy example — continuing block 5

$\boldsymbol{a}_d = [-0.15, 0.30, 0.55]$, $m = 1.5$, $\boldsymbol{g} = [0, 0, -9.81]$, $\psi_d = 0$:

$$\boldsymbol{F}_d = 1.5\,[-0.15,\ 0.30,\ 10.36] = [-0.225,\ 0.450,\ 15.540]\ \text{N}, \qquad \|\boldsymbol{F}_d\| = 15.548\ \text{N},$$

$$\boldsymbol{b}_{3d} = [-0.0145,\ 0.0289,\ 0.9995], \qquad R_d = \begin{bmatrix} 0.9999 & 0.0000 & -0.0145 \\ 0.0004 & 0.9996 & 0.0289 \\ 0.0145 & -0.0289 & 0.9995 \end{bmatrix}.$$

Tilt $= \arccos(0.9995) = 1.85°$: lean slightly towards $-x$, $+y$ and push 15.5 N (more than the 14.7 N weight, because it must also climb). Check: $\|R_d^\top R_d - I\| = 1.1 \times 10^{-16}$, $\det R_d = 1$.

In our drone project

A drone moves sideways by *leaning*, like a person pushing a heavy cart. This block converts "accelerate this way" into "lean this much in this direction, and push this hard". It is `compute_desired_force()` and `compute_desired_attitude()` in `so3_controller.py`. The attitude is built as a rotation matrix, never as roll/pitch/yaw angles, so there is no gimbal lock.

Toy model: `step6_force_attitude.m`, `lib/force_attitude.m`.

4.7 Block 7 — Attitude control on SO(3) (inner loop)

Equations M1.20 – M1.23 Geometric attitude controller

$$\boldsymbol{e}_R = \tfrac{1}{2}\left(R_d^\top \hat{R} - \hat{R}^\top R_d\right)^\vee \tag{M1.20}$$

$$\boldsymbol{e}_\omega = \hat{\boldsymbol{\omega}} - \hat{R}^\top R_d\, \boldsymbol{\omega}_d \tag{M1.21}$$

$$\boldsymbol{\tau} = -K_R \boldsymbol{e}_R - K_\omega \boldsymbol{e}_\omega + \hat{\boldsymbol{\omega}} \times J\hat{\boldsymbol{\omega}} - J\left([\hat{\boldsymbol{\omega}}]_\times \hat{R}^\top R_d \boldsymbol{\omega}_d - \hat{R}^\top R_d \dot{\boldsymbol{\omega}}_d\right) \tag{M1.22}$$

$$T = \boldsymbol{F}_d \cdot \hat{R}\, \boldsymbol{e}_3 \tag{M1.23}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{e}_R$	attitude error ("≈ rotation vector" for small errors)	$\mathbb{R}^3$	rad (small angles)
$\boldsymbol{e}_\omega$	angular-velocity error (body frame)	$\mathbb{R}^3$	rad/s
$\boldsymbol{\omega}_d$, $\dot{\boldsymbol{\omega}}_d$	desired body rate and its derivative	$\mathbb{R}^3$	rad/s, rad/s^2
K_R, K_ω	attitude and rate gains	$\mathbb{R}^{3\times3}$	N m, N m s
$\boldsymbol{\tau}$, T	torque and thrust commands	$\mathbb{R}^3$, $\mathbb{R}$	N m, N

Derivation of $\boldsymbol{e}_R$ (M1.20)

Measure "how far" $\hat{R}$ is from R_d with the error function

$$\Psi(\hat{R}, R_d) = \tfrac{1}{2}\operatorname{tr}\left(I - R_d^\top \hat{R}\right) = 1 - \cos\varphi \quad \in [0, 2],$$

where φ is the angle of the relative rotation $R_d^\top \hat{R}$ (because $\operatorname{tr} R = 1 + 2\cos\varphi$). $\Psi = 0$ exactly when $\hat{R} = R_d$. Now rotate the drone slightly, $\hat{R} \to \hat{R}\mathrm{Exp}(\epsilon\boldsymbol{\eta})$, and differentiate:

$$\frac{d}{d\epsilon}\Psi\Big|_{\epsilon=0} = -\tfrac{1}{2}\operatorname{tr}\left(R_d^\top \hat{R}[\boldsymbol{\eta}]_\times\right) \overset{(4)}{=} \tfrac{1}{2}\boldsymbol{\eta}^\top\left(R_d^\top \hat{R} - \hat{R}^\top R_d\right)^\vee = \boldsymbol{e}_R \cdot \boldsymbol{\eta}.$$

So $\boldsymbol{e}_R$ is the *gradient* of the attitude error on SO(3): it points in the direction of rotation that increases the error fastest, and the controller pushes the opposite way. For a single-axis error of angle φ about $\boldsymbol{n}$, $\boldsymbol{e}_R = \sin\varphi\, \boldsymbol{n}$, which is $\approx \varphi\boldsymbol{n}$ for small angles.

Derivation of e_ω (M1.21)

Let $R_e = R_d^\top \hat{R}$ be the relative rotation. Using $\dot{\hat{R}} = \hat{R}[\hat{\boldsymbol{\omega}}]_\times$, $\dot{R}_d = R_d[\boldsymbol{\omega}_d]_\times$ and $[\boldsymbol{\omega}_d]_\times R_e = R_e[R_e^\top \boldsymbol{\omega}_d]_\times$ (from (3)),

$$\dot{R}_e = R_d^\top \hat{R}[\hat{\boldsymbol{\omega}}]_\times - [\boldsymbol{\omega}_d]_\times R_d^\top \hat{R} = R_e\,[\hat{\boldsymbol{\omega}} - \hat{R}^\top R_d \boldsymbol{\omega}_d]_\times = R_e\,[\boldsymbol{e}_\omega]_\times.$$

$\boldsymbol{e}_\omega$ is therefore the angular velocity of the *error* rotation, expressed in the body frame: how fast the drone is turning relative to where it should be turning. Note the sign: *actual minus desired*.

Derivation of the torque law (M1.22) by Lyapunov's method

Differentiate $\boldsymbol{e}_\omega$. With $\frac{d}{dt}(\hat{R}^\top R_d) = -[\hat{\boldsymbol{\omega}}]_\times \hat{R}^\top R_d + \hat{R}^\top R_d [\boldsymbol{\omega}_d]_\times$ and $[\boldsymbol{\omega}_d]_\times \boldsymbol{\omega}_d = 0$,

$$J\dot{\boldsymbol{e}}_\omega = J\dot{\hat{\boldsymbol{\omega}}} + J[\hat{\boldsymbol{\omega}}]_\times \hat{R}^\top R_d \boldsymbol{\omega}_d - J\hat{R}^\top R_d \dot{\boldsymbol{\omega}}_d \overset{(M1.4)}{=} \boldsymbol{\tau} - \hat{\boldsymbol{\omega}} \times J\hat{\boldsymbol{\omega}} + J([\hat{\boldsymbol{\omega}}]_\times \hat{R}^\top R_d \boldsymbol{\omega}_d - \hat{R}^\top R_d \dot{\boldsymbol{\omega}}_d).$$

Substituting (M1.22), every nonlinear term cancels and the error dynamics become

$$J\dot{\boldsymbol{e}}_\omega = -K_R \boldsymbol{e}_R - K_\omega \boldsymbol{e}_\omega.$$

Take $K_R = k_R I$, $K_\omega = k_\omega I$ and the energy-like function $V = \frac{1}{2}\boldsymbol{e}_\omega^\top J \boldsymbol{e}_\omega + k_R \Psi$. Using $\dot{\Psi} = \boldsymbol{e}_R \cdot \boldsymbol{e}_\omega$ (same calculation as above, with $\boldsymbol{\eta} = \boldsymbol{e}_\omega$):

$$\dot{V} = \boldsymbol{e}_\omega^\top(-k_R \boldsymbol{e}_R - k_\omega \boldsymbol{e}_\omega) + k_R\, \boldsymbol{e}_R \cdot \boldsymbol{e}_\omega = -k_\omega \|\boldsymbol{e}_\omega\|^2 \le 0.$$

V never increases. LaSalle's invariance principle then shows that $(\boldsymbol{e}_R, \boldsymbol{e}_\omega) \to 0$ from every attitude with $\Psi < 2$, i.e. from everything except exactly upside-down. Unlike Euler-angle controllers, there is no singularity.

Simplified form. Near the target ($\hat{R}^\top R_d \approx I$) and for slow manoeuvres, the last bracket reduces to $-J([\hat{\boldsymbol{\omega}}]_\times \boldsymbol{\omega}_d - \dot{\boldsymbol{\omega}}_d) \approx +J\dot{\boldsymbol{\omega}}_d$, which is the term printed in the original image.

Derivation of the thrust (M1.23)

Thrust acts only along the *current* axis $\hat{R}\boldsymbol{e}_3$. Its useful part is the projection of the desired force on that axis, $T = \boldsymbol{F}_d \cdot \hat{R}\boldsymbol{e}_3$. When the attitude has converged ($\hat{R}\boldsymbol{e}_3 = \boldsymbol{b}_{3d}$), $T = \|\boldsymbol{F}_d\|$. While still tilted the wrong way, T is automatically reduced, so the drone does not push hard in a wrong direction.

Toy example — recovering from a 10° roll

$\hat{R} = \mathrm{Exp}([0.1745, 0, 0])$ (10° roll), $\hat{\boldsymbol{\omega}} = [0.2, 0, 0]$ rad/s (still rolling further), $R_d = I$, $\boldsymbol{\omega}_d = \dot{\boldsymbol{\omega}}_d = 0$, $\boldsymbol{F}_d = [0, 0, 14.715]$:

$$\boldsymbol{e}_R = \tfrac{1}{2}(\hat{R} - \hat{R}^\top)^\vee = [\sin 10°, 0, 0] = [0.1736, 0, 0], \qquad \boldsymbol{e}_\omega = [0.2, 0, 0],$$

$$\boldsymbol{\tau} = -\begin{bmatrix} 4(0.1736) \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0.8(0.2) \\ 0 \\ 0 \end{bmatrix} + \underbrace{\hat{\boldsymbol{\omega}} \times J\hat{\boldsymbol{\omega}}}_{=0} = \begin{bmatrix} -0.8546 \\ 0 \\ 0 \end{bmatrix} \text{N m}, \qquad T = 14.715 \cos 10° = 14.491 \text{ N}.$$

A restoring torque about $-x$ rolls it back, and thrust is reduced to the useful vertical part.

Correction to the original flow image — sign of the rate error

The image prints $\boldsymbol{e}_\omega = \boldsymbol{\omega}_d - \hat{\boldsymbol{\omega}}$ together with $\boldsymbol{\tau} = -K_\omega \boldsymbol{e}_\omega + \ldots$, i.e. $\boldsymbol{\tau} = +K_\omega \hat{\boldsymbol{\omega}} + \ldots$. That is negative damping. With it, $\dot{V} = +k_\omega \|\boldsymbol{e}_\omega\|^2$ and the energy *grows*. The toy model starts 60° from level: with the corrected sign the error decays to 2×10^{-6} deg within 3 s and V decreases

monotonically. With the image's sign the drone exceeds 50 rad/s (tumbling) after 0.066 s (fig. 4).

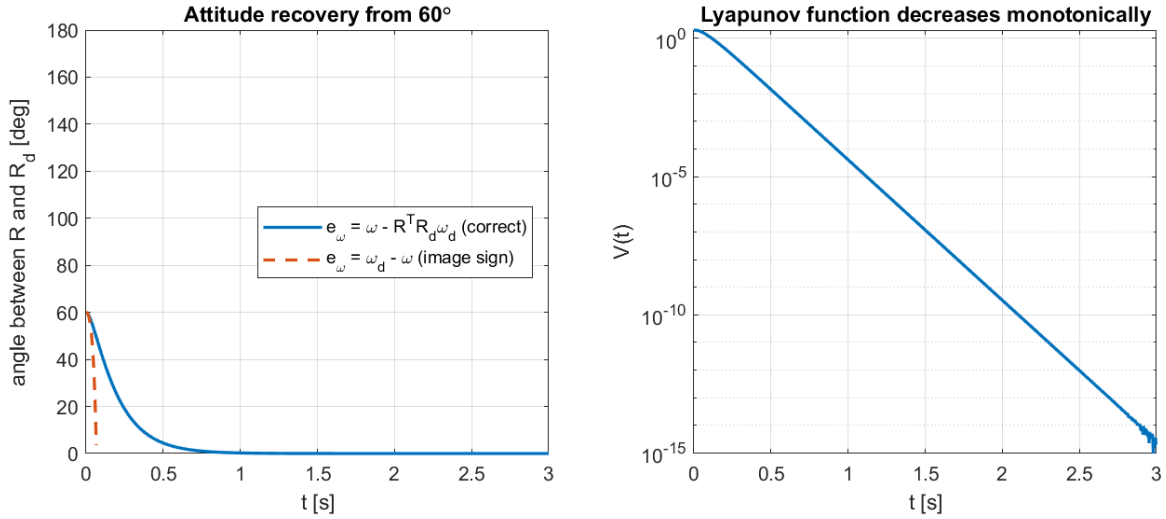


Figure 4: Toy model, Model 1 step 7. Left: recovery from 60° with the corrected and the original sign of $\boldsymbol{e}_\omega$. Right: the Lyapunov function V decreases monotonically with the corrected law.

In our drone project

This is the fast inner loop: "I should be leaning like R_d but I am leaning like $\hat{R}$, so which way do I twist?". It works directly with rotation matrices, so it handles any orientation without gimbal lock. That is the central claim of our README ("Euler-angle-free"). In code it is `compute_attitude_error()`, `compute_angular_velocity_error()` and `compute_moment()` in `so3_controller.py`. In the current ROS 2 interface the node publishes $\boldsymbol{F}_d$, $\boldsymbol{e}_R$ and $\boldsymbol{\tau}$ as diagnostics and sends velocity commands, and ArduPilot executes its own inner loop. The equations are the same.

Toy model: `step7_attitude_control.m`, `lib/attitude_control.m`.

4.8 Block 8 — Actuator commands and feedback

Equations M1.24 – M1.25 Control allocation and motor speeds

$$\begin{bmatrix} T \\ \boldsymbol{\tau} \end{bmatrix} = \Gamma\, \boldsymbol{f}, \qquad \Gamma = \begin{bmatrix} 1 & 1 & 1 & 1 \\ y_1 & y_2 & y_3 & y_4 \\ -x_1 & -x_2 & -x_3 & -x_4 \\ \kappa_1 c_m & \kappa_2 c_m & \kappa_3 c_m & \kappa_4 c_m \end{bmatrix} \tag{M1.24}$$

$$\Omega_i = \sqrt{\frac{(\Gamma^{-1}\boldsymbol{u})_i}{k_f}} \quad \text{clipped to } [\Omega_{\min}, \Omega_{\max}] \tag{M1.25}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{f} = [f_1..f_4]$	individual rotor thrusts	$\mathbb{R}^4$	N
(x_i, y_i)	rotor position in the body frame	$\mathbb{R}^2$	m
κ_i	spin sign (-1 CCW, $+1$ CW)	$\{\pm 1\}$	–
c_m	yaw torque per newton of thrust	$\mathbb{R}$	m
Γ	allocation (mixer) matrix	$\mathbb{R}^{4\times 4}$	–, m
k_f	thrust constant	$\mathbb{R}$	$\mathrm{N\,s^2}$
$\boldsymbol{\Omega}$	rotor speeds	$\mathbb{R}^4$	rad/s

Derivation

Each rotor i produces a force $f_i \boldsymbol{e}_3$ at $\boldsymbol{r}_i = [x_i, y_i, 0]^\top$. Summing:

- total thrust $T = \sum_i f_i$;
- torque $\sum_i \boldsymbol{r}_i \times f_i \boldsymbol{e}_3 = \sum_i f_i [y_i, -x_i, 0]^\top$, which gives the roll and pitch rows;
- yaw: the air resists the spinning propeller, so the motor reacts on the body with $\kappa_i c_m f_i$ about $\boldsymbol{e}_3$. CCW rotors push the body clockwise.

This gives (M1.24). Momentum theory gives a rotor thrust proportional to the square of the tip speed, $f_i = k_f \Omega_i^2$. Inverting both relations gives (M1.25). Motors can neither reverse nor exceed their maximum speed, so Ω_i is clipped, and the wrench actually produced is $\Gamma k_f \boldsymbol{\Omega}^2$.

For the Iris (ArduPilot Quad-X: 1 front-right CCW, 2 back-left CCW, 3 front-left CW, 4 back-right CW):

$$\Gamma = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -0.22 & 0.22 & 0.22 & -0.22 \\ -0.13 & 0.13 & -0.13 & 0.13 \\ -0.016 & -0.016 & 0.016 & 0.016 \end{bmatrix}.$$

Toy example — hover and the roll correction of block 7

Hover $\boldsymbol{u} = [14.715, 0, 0, 0]$: $f_i = 14.715/4 = 3.679\,\mathrm{N}$, $\Omega_i = \sqrt{3.679/8.549 \times 10^{-6}} = 656\,\mathrm{rad/s} = 6264\,\mathrm{rpm}$.
Roll correction $\boldsymbol{u} = [14.491, -0.8546, 0, 0]$: by symmetry $f_1 = f_4 = a$ and $f_2 = f_3 = b$, with $2a + 2b = 14.491$ and $0.44(b - a) = -0.8546$. So $a = 4.594\,\mathrm{N}$ and $b = 2.652\,\mathrm{N}$, i.e.
$\boldsymbol{\Omega} = [733, 557, 557, 733]\,\mathrm{rad/s}$. The right-side rotors (1, 4) speed up and the drone rolls back to level.

In our drone project

The flight controller does not "send a torque". It sends four motor speeds. This block is ArduPilot's motor mixer. When our node commands velocities (`/ap/v1/cmd_vel`), ArduPilot runs blocks 5–8 internally. When it commands force and moment, this is the matrix that turns them into ESC signals. The closed loop in fig. 5 is the whole of Model 1 working together: the arrow from block 8 back to block 1.

Toy model: `step8_actuators_feedback.m`, `lib/motor_allocation.m`.

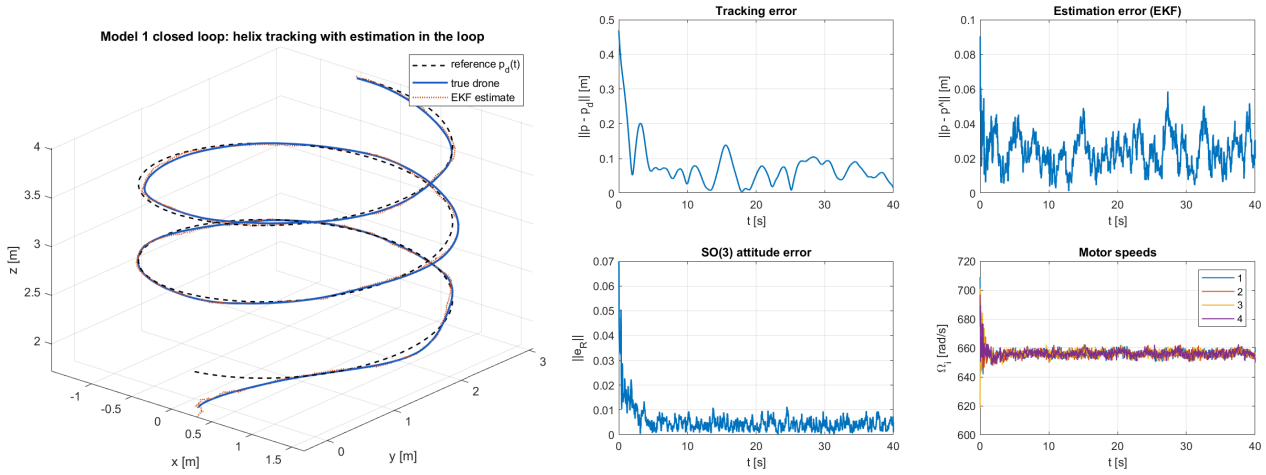


Figure 5: Toy model, Model 1 step 8: the complete loop (truth → sensors → EKF → blocks 5–7 → allocation → truth) tracking a climbing helix in wind. After the transient: tracking RMSE 6.8 cm, estimation RMSE 2.6 cm, motor speeds 619–709 rad/s.

4.9 Recap of Model 1

#	Block	In	Out	The idea in one sentence
1	Nonlinear dynamics	thrust T, torque $\boldsymbol{\tau}$	motion $\boldsymbol{p}, \boldsymbol{v}, R, \boldsymbol{\omega}$	Newton and Euler: push along the body z-axis, gravity pulls down, torque spins the body.
2	Linearization	operating point	A, B	Near any flight condition the drone behaves like a linear system.
3	Error dynamics	A, B, Δt	F	Small errors evolve linearly; without feedback they grow.
4	Error-state EKF	$\boldsymbol{u}_k$, sensors $\boldsymbol{z}_k$	estimate $\hat{\boldsymbol{x}}_k$, P_k	Predict with physics, correct with camera and IMU, weighting by trust.
5	Position control	$\hat{\boldsymbol{p}}, \hat{\boldsymbol{v}}$, reference	$\boldsymbol{a}_d$	Accelerate towards the target like a spring with a damper.
6	Force & attitude	$\boldsymbol{a}_d$, yaw	$\boldsymbol{F}_d$, R_d	To accelerate sideways, lean; thrust direction = desired force direction.
7	SO(3) attitude	$\hat{R}, \hat{\boldsymbol{\omega}}, R_d, \boldsymbol{F}_d$	$\boldsymbol{\tau}$, T	Twist along the gradient of the attitude error; energy always decreases.
8	Allocation	T, $\boldsymbol{\tau}$	$\Omega_1..\Omega_4$	Split thrust and torque among four rotors; speed $\propto \sqrt{\text{thrust}}$.

5 Model 2 — Data-Driven Model and Optimal Control

Model 1 needs the physics: m, J, rotor constants. Model 2 asks a different question: *if we only had recorded flight data, could we still build a good controller?* It learns a discrete-time linear model from data (blocks 1–2). It then designs an optimal controller on that model in two ways: LQR, which is optimal and simple but unaware of limits (blocks 3–6), and MPC, which is optimal and respects limits but needs an optimizer (blocks 7–9). It flies the real, nonlinear drone with the result (block 10). Finally an extended Kalman filter (block 11) builds the state that the controllers need from camera pixels and the gyroscope, using the same learned model. That closes the loop.

All matrices here are *discrete-time*: $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ with $\Delta t = 0.04\,\text{s}$, which is the 25 Hz `control_rate` of `controller.yaml`.

5.1 Block 1 — Nonlinear flight data

Equation M2.1 Sampled state and input (deviation from hover)

$$\boldsymbol{x}_k = \begin{bmatrix} \boldsymbol{p}(t_k) - \boldsymbol{p}_{\text{ref}} \\ \boldsymbol{v}(t_k) \\ \text{Log}(R(t_k)) \\ \boldsymbol{\omega}(t_k) \end{bmatrix} \in \mathbb{R}^{12}, \qquad \boldsymbol{u}_k = \begin{bmatrix} T(t_k) - mg_0 \\ \boldsymbol{\tau}(t_k) \end{bmatrix} \in \mathbb{R}^4, \qquad t_k = k\,\Delta t. \tag{M2.1}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{x}_k$	state deviation at sample k	$\mathbb{R}^{12}$	m, m/s, rad, rad/s
$\boldsymbol{u}_k$	input deviation at sample k	$\mathbb{R}^4$	N, N m
$\boldsymbol{p}_{\text{ref}}$	hover point	$\mathbb{R}^3$	m ($[0, 0, 2]$)
Δt	sample time	$\mathbb{R}$	s (0.04)
N	number of samples in a flight	$\mathbb{N}$	– (625 per 25 s flight)

Derivation and reasoning

Operating point. Hover is an equilibrium of (M1.1)–(M1.4): with $\boldsymbol{v} = 0$, $R = I$, $\boldsymbol{\omega} = 0$, $T = mg_0$, $\boldsymbol{\tau} = 0$, every derivative is zero. Linearizing about it gives $\delta\dot{\boldsymbol{x}} = A_c\delta\boldsymbol{x} + B_c\delta\boldsymbol{u}$ with *no constant term*. Measuring the state and input as deviations from hover is what lets the pure linear form $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ fit the data. If the raw thrust T were used, gravity would appear as a constant offset that this form cannot represent (correction 6 in section 2.5).

Attitude as a vector. DMDc needs vectors. $\text{Log}(R)$ turns the 3×3 rotation into the 3-vector rotation angle, which near hover is the familiar (roll, pitch, yaw).

Sampling. The computer applies $\boldsymbol{u}_k$ and holds it for Δt (zero-order hold). Hence the natural model is discrete.

Persistent excitation. A free quadrotor is unstable (section 4.3), so data is always recorded with some stabilising pilot in the loop, here a simple PD controller (the role ArduPilot plays). If the input were *only* feedback, $\boldsymbol{u}_k = -K_0\boldsymbol{x}_k$, the rows of U would be linear combinations of the rows of X. The data could not separate "the drone" from "the pilot". A small random excitation is therefore added on top (± 2 N thrust, ± 0.03 N m torque, held for 0.2 s), which makes the stacked data matrix full rank.

Toy example — the recorded data set

Five flights of 25 s at 25 Hz, each starting from a random point near $[0, 0, 2]$ m: 5×625 samples of $\boldsymbol{x}_k \in \mathbb{R}^{12}$ and $\boldsymbol{u}_k \in \mathbb{R}^4$, with small logging noise. Flights 1–4 are used for identification and flight 5 is kept for validation. The largest roll/pitch in the data is 6.5°, well inside the near-linear range of section 4.2.

In our drone project

In the project, this block is a `ros2 bag record` of `/ap/v1/pose/filtered`, `/ap/v1/twist/filtered` and the commands we send, recorded from ArduPilot SITL in Gazebo or from the real drone. Nothing about mass or inertia needs to be known.

Toy model: `model2_dmdc_lqr_mpc/step1_flight_data.m`, `lib/state_to_dev.m`, `lib/baseline_pilot.m`.

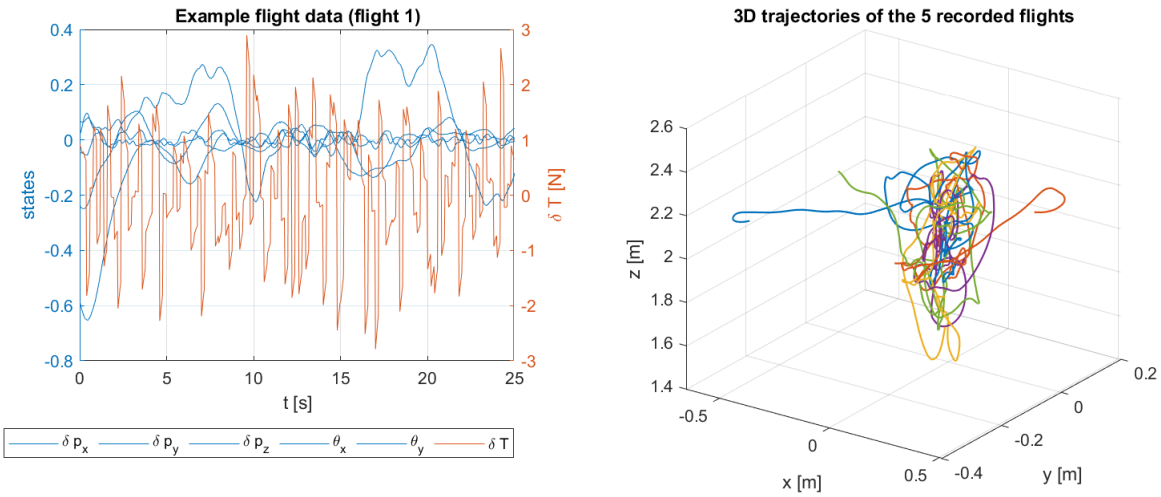


Figure 6: Toy model, Model 2 step 1: example time histories and the 3D tracks of the five recorded flights.

5.2 Block 2 — DMDc: Dynamic Mode Decomposition with control

Equations M2.2 – M2.4 Snapshot matrices and least-squares model

$$X = [\boldsymbol{x}_1 \ \cdots \ \boldsymbol{x}_{N-1}], \quad X^+ = [\boldsymbol{x}_2 \ \cdots \ \boldsymbol{x}_N], \quad U = [\boldsymbol{u}_1 \ \cdots \ \boldsymbol{u}_{N-1}] \tag{M2.2}$$

$$[\,A \ \ B\,] = X^+ \begin{bmatrix} X \\ U \end{bmatrix}^{\dagger} \tag{M2.3}$$

$$\boldsymbol{x}_{k+1} \approx A\,\boldsymbol{x}_k + B\,\boldsymbol{u}_k \tag{M2.4}$$

Symbol	Meaning	Dimension	Unit
$X,\ X^+$	state snapshots and the same shifted by one sample	$\mathbb{R}^{12\times(N-1)}$	mixed
U	input snapshots	$\mathbb{R}^{4\times(N-1)}$	N, N m
$\Omega = [X;U]$	stacked regressor matrix	$\mathbb{R}^{16\times(N-1)}$	mixed
$\Omega^{\dagger}$	pseudo-inverse of Ω	$\mathbb{R}^{(N-1)\times16}$	mixed
A	learned discrete state matrix	$\mathbb{R}^{12\times12}$	–
B	learned discrete input matrix	$\mathbb{R}^{12\times4}$	mixed

Derivation

We want the pair $G = [A\ B]$ for which $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k = G\,[\boldsymbol{x}_k;\boldsymbol{u}_k]$ holds as well as possible for *every* recorded sample. Written for all samples at once, this is $X^+ \approx G\,\Omega$. The least-squares choice minimises the sum of squared one-step prediction errors:

$$\min_{G} \ \|X^+ - G\,\Omega\|_F^2 .$$

Setting the gradient to zero, $-2\,(X^+ - G\Omega)\Omega^{\top} = 0$, gives the *normal equations* $G\,\Omega\Omega^{\top} = X^+\Omega^{\top}$. If Ω has full row rank (16, guaranteed by the excitation), the 16×16 matrix $\Omega\Omega^{\top}$ is invertible and

$$G = X^+\Omega^{\top}\left(\Omega\Omega^{\top}\right)^{-1} = X^+\Omega^{\dagger}.$$

In practice $\Omega^{\dagger}$ is computed from the SVD $\Omega = U_\Omega\Sigma V_\Omega^{\top}$ as $V_\Omega\Sigma^{-1}U_\Omega^{\top}$, and tiny singular values can be truncated to suppress noise (Proctor, Brunton & Kutz, 2016). Splitting the columns of G gives A (first 12) and B (last 4).

Relation to Model 1. If the data came from the linearized physics, the exact answer would be the zero-order-hold discretization of (8): $A = e^{A_c \Delta t}$, $B = \int_0^{\Delta t} e^{A_c s} ds\, B_c$. DMDc recovers these numbers from data alone.

Toy example — a scalar system by hand

True system $x_{k+1} = 0.9x_k + 0.5u_k$. Three recorded steps: $X = [1,\ 1.4,\ 0.76]$, $X^+ = [1.4,\ 0.76,\ 0.934]$, $U = [1,\ -1,\ 0.5]$.

$$\Omega\Omega^\top = \begin{bmatrix} 3.5376 & -0.02 \\ -0.02 & 2.25 \end{bmatrix}, \qquad X^+\Omega^\top = \begin{bmatrix} 3.17384 & 1.107 \end{bmatrix},$$

$$[a \quad b] = \frac{1}{7.9592} \begin{bmatrix} 3.17384 & 1.107 \end{bmatrix} \begin{bmatrix} 2.25 & 0.02 \\ 0.02 & 3.5376 \end{bmatrix} = \begin{bmatrix} 0.9000 & 0.5000 \end{bmatrix}.$$

The true model is recovered exactly, because the data is noise-free.
The drone (2500 snapshots, $\operatorname{rank}[X; U] = 16$):

entry	physical meaning	DMDc	physics (ZOH)
$A_{1,4}$	Δt (position from velocity)	0.0399	0.0400
$A_{4,8}$	$\approx g_0 \Delta t$ (v_x from pitch)	0.3909	0.3924
$A_{5,7}$	$\approx -g_0 \Delta t$ (v_y from roll)	−0.3862	−0.3924
$B_{6,1}$	$\Delta t/m$ (v_z from thrust)	0.0268	0.0267
$B_{10,2}$	$\Delta t/J_x$ (roll rate from roll torque)	2.0007	2.0000
$B_{12,4}$	$\Delta t/J_z$ (yaw rate from yaw torque)	1.0096	1.0000

Overall $\|A - A_{\text{phys}}\|/\|A_{\text{phys}}\| = 0.95\%$ and $\|B - B_{\text{phys}}\|/\|B_{\text{phys}}\| = 0.81\%$. On the unseen flight, the 1-second-ahead position prediction error is 1.21 cm, compared with 1.14 cm for the exact physics model. The learned mass is $\Delta t/B_{6,1} = 1.49$ kg and the learned roll inertia is $\Delta t/B_{10,2} = 0.0200\,\text{kg}\,\text{m}^2$.

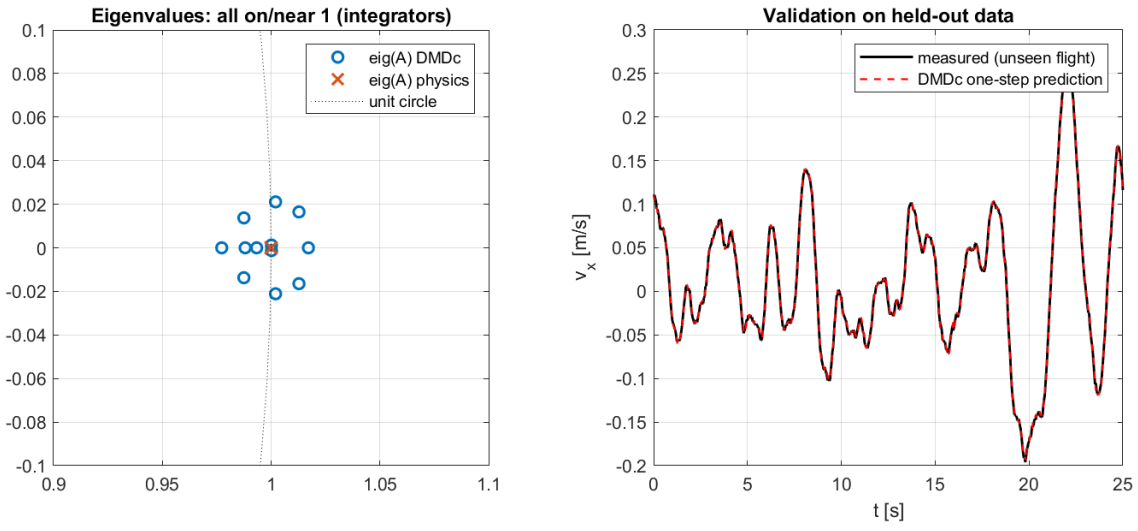


Figure 7: Toy model, Model 2 step 2. Left: eigenvalues of the learned and physical A, all at or near 1 (integrators). Right: one-step predictions on a flight that was *not* used for learning.

In our drone project

DMDc is "curve fitting for dynamics". It looks at thousands of moments of the form "the drone was here, we pushed like this, and 40 ms later it was there", and finds the single linear rule that explains them best. For our drone this means we can re-identify the model whenever something

changes (a heavier camera, a new battery, a different frame) simply by flying for two minutes.

Toy model: `step2_dmdc.m`, `lib/dmdc.m`, `lib/c2d_zoh.m`.

5.3 Block 3 — Bellman equation (optimal control)

Equations M2.5 – M2.7 Cost, Bellman equation, quadratic value function

$$J = \sum_{k=0}^{\infty} \left(\boldsymbol{x}_k^\top Q\, \boldsymbol{x}_k + \boldsymbol{u}_k^\top R\, \boldsymbol{u}_k \right) \tag{M2.5}$$

$$V(\boldsymbol{x}_k) = \min_{\boldsymbol{u}_k} \left[\boldsymbol{x}_k^\top Q \boldsymbol{x}_k + \boldsymbol{u}_k^\top R \boldsymbol{u}_k + V(\boldsymbol{x}_{k+1}) \right], \qquad \boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k \tag{M2.6}$$

$$V(\boldsymbol{x}) = \boldsymbol{x}^\top P\, \boldsymbol{x} \tag{M2.7}$$

Symbol	Meaning	Dimension	Unit
J	total cost of a flight	$\mathbb{R}$	– (dimensionless)
Q	state weight, $Q = Q^\top \succeq 0$	$\mathbb{R}^{12\times 12}$	$1/(\text{unit of } x_i)^2$
R	input weight, $R = R^\top \succ 0$	$\mathbb{R}^{4\times 4}$	$1/\text{N}^2$, $1/(\text{N m})^2$
$V(\boldsymbol{x})$	value function: lowest possible cost from state $\boldsymbol{x}$	$\mathbb{R}$	–
P	value matrix, $P = P^\top \succeq 0$	$\mathbb{R}^{12\times 12}$	as Q

Derivation

(M2.5) The cost. $\boldsymbol{x}^\top Q \boldsymbol{x}$ is a weighted sum of squared errors and $\boldsymbol{u}^\top R \boldsymbol{u}$ a weighted sum of squared effort. Minimising J trades accuracy against effort. We choose the weights by *Bryson's rule*, $Q_{ii} = 1/x_{i,\max}^2$ and $R_{jj} = 1/u_{j,\max}^2$, where $x_{i,\max}$ is the largest acceptable deviation. A 50 cm position error then costs as much as a 1 m/s speed error, a 0.2 rad tilt, or 5 N of extra thrust.

(M2.6) Principle of optimality (Bellman). Split the infinite sum into the first step and the rest:

$$V(\boldsymbol{x}_k) = \min_{\boldsymbol{u}_k, \boldsymbol{u}_{k+1}, \ldots} \left[\ell(\boldsymbol{x}_k, \boldsymbol{u}_k) + \sum_{j>k} \ell(\boldsymbol{x}_j, \boldsymbol{u}_j) \right] = \min_{\boldsymbol{u}_k} \Big[\ell(\boldsymbol{x}_k, \boldsymbol{u}_k) + \underbrace{\min_{\boldsymbol{u}_{k+1}, \ldots} \sum_{j>k} \ell(\boldsymbol{x}_j, \boldsymbol{u}_j)}_{V(\boldsymbol{x}_{k+1})} \Big].$$

Whatever the first move is, the remaining moves must be optimal for the state it leads to. Otherwise we could improve them and lower the total. The infinite-dimensional problem collapses to a one-step problem.

(M2.7) Quadratic value function. Suppose $V(\boldsymbol{x}_{k+1}) = \boldsymbol{x}_{k+1}^\top P \boldsymbol{x}_{k+1}$. The bracket in (M2.6) is then a quadratic function of $\boldsymbol{u}_k$; its minimiser is linear in $\boldsymbol{x}_k$ (block 4), and the minimum value is again a quadratic form in $\boldsymbol{x}_k$ (block 5). The quadratic shape is preserved by the Bellman equation, so the guess $V = \boldsymbol{x}^\top P \boldsymbol{x}$ is self-consistent, and all that remains is to find P.

Toy example — one Bellman step by hand

$x_{k+1} = 1.1x_k + 0.5u_k$ (unstable: grows 10% per step), $q = r = 1$, next-step value $V(x) = 1 \cdot x^2$, current $x = 1$:

$$\text{RHS}(u) = 1 + u^2 + (1.1 + 0.5u)^2, \qquad \frac{d}{du} = 2u + (1.1 + 0.5u) = 0 \;\Rightarrow\; u^* = -\frac{1.1}{2.5} = -0.44,$$

$$V(1) = 1 + 0.1936 + (1.1 - 0.22)^2 = 1.968.$$

`fminsearch` on the same function returns $u^* = -0.4400$ and $V = 1.9680$.

Drone (DMDc model, $V_{\text{next}} = \boldsymbol{x}^\top Q\boldsymbol{x}$, drone 50/30/20 cm off target): brute-force minimisation over $\boldsymbol{u} \in \mathbb{R}^4$ and the closed form both give $\boldsymbol{u}^* = [-0.0104, -0.0001, 0.0001, 0.0000]$ and the same value 3.0393. Doing nothing ($\boldsymbol{u} = 0$) costs 14 593 after 10 s and keeps growing.

In our drone project

Q and R are where *we* write down what "good flying" means for our mission. For mapping with a camera we care about smooth, gentle motion (blur-free images), so tilt and rate errors are weighted heavily and speed is limited to the `max_velocity` of 1 m/s. The Bellman equation turns that wish list into a mathematical problem with a unique best answer.

Toy model: `step3_bellman_cost.m`, `lib/bellman_backup.m`.

5.4 Block 4 — Linear Quadratic Regulator (LQR)

Equations M2.8 – M2.9 Optimal gain and control law

$$K = (R + B^\top P B)^{-1} B^\top P A \tag{M2.8}$$

$$\boldsymbol{u}_k = -K\,\boldsymbol{x}_k \tag{M2.9}$$

Symbol	Meaning	Dimension	Unit
K	state-feedback gain	$\mathbb{R}^{4\times 12}$	N/m, N s/m, N m/rad, …
$R + B^\top P B$	curvature of the one-step cost in $\boldsymbol{u}$	$\mathbb{R}^{4\times 4}$	–

Derivation

Insert $V(\boldsymbol{x}_{k+1}) = (A\boldsymbol{x} + B\boldsymbol{u})^\top P (A\boldsymbol{x} + B\boldsymbol{u})$ into the bracket of (M2.6) and differentiate with respect to $\boldsymbol{u}$ (using $\partial(\boldsymbol{u}^\top M \boldsymbol{u})/\partial \boldsymbol{u} = 2M\boldsymbol{u}$ for symmetric M):

$$\frac{\partial}{\partial \boldsymbol{u}}\Big[\boldsymbol{x}^\top Q\boldsymbol{x} + \boldsymbol{u}^\top R\boldsymbol{u} + (A\boldsymbol{x} + B\boldsymbol{u})^\top P (A\boldsymbol{x} + B\boldsymbol{u})\Big] = 2R\boldsymbol{u} + 2B^\top P(A\boldsymbol{x} + B\boldsymbol{u}) = \boldsymbol{0}.$$

Collecting the $\boldsymbol{u}$ terms: $(R + B^\top P B)\,\boldsymbol{u} = -B^\top P A\,\boldsymbol{x}$. Because $R \succ 0$ and $P \succeq 0$, the matrix $R + B^\top P B$ is positive definite. It is therefore invertible, and its positive curvature (the Hessian $2(R + B^\top P B)$) guarantees a minimum. Hence $\boldsymbol{u}^* = -(R + B^\top P B)^{-1} B^\top P A\,\boldsymbol{x} = -K\boldsymbol{x}$.

Toy example — the scalar system

With the converged $P = 3.1215$ (block 6): $K = \dfrac{ab\,P}{r + b^2 P} = \dfrac{1.1 \cdot 0.5 \cdot 3.1215}{1 + 0.25 \cdot 3.1215} = 0.9643$. Closed loop: $x_{k+1} = (a - bK)x_k = 0.6179\,x_k$. Without control the state grows 10% per step; with LQR it shrinks 38% per step.

Drone gain (from block 6), two of its 48 numbers:

$$\delta T = -9.04\,\delta z - 6.92\,v_z, \qquad \tau_x = 0.273\,\delta y + 0.321\,v_y - 1.444\,\theta_x - 0.276\,\omega_x.$$

If the drone is too high, thrust drops. If it is to the left ($\delta y > 0$), it rolls right ($+\tau_x$, which makes $\dot{v}_y < 0$), but less so if it is already rolled or rolling.

With only a one-step look-ahead (P = one Bellman step from Q), the gain gives a spectral radius $\rho(A - BK) = 1.0024 > 1$: *unstable*. The full infinite-horizon P is essential.

In our drone project

The LQR controller is just a table: a 4×12 matrix K. Every 40 ms it multiplies the 12 measured errors by K and obtains thrust and three torques. It is extremely cheap to run on the drone and, for the cost we chose, provably the best possible linear controller. Its weakness is that it knows nothing about limits.

Toy model: `step4_lqr_gain.m`, `lib/lqr_gain.m`.

5.5 Block 5 — Riccati recursion

Equation M2.10 Discrete Riccati difference equation

$$P_{k+1} = A^\top P_k A - A^\top P_k B (R + B^\top P_k B)^{-1} B^\top P_k A + Q, \qquad P_0 = Q \text{ (or any terminal weight)} \tag{M2.10}$$

Symbol	Meaning	Dimension	Unit
P_k	value matrix of a problem with k steps to go	$\mathbb{R}^{12\times 12}$	as Q
k	iteration index (steps-to-go, *not* time)	$\mathbb{N}$	–

Derivation

Substitute the optimal input $\boldsymbol{u}^* = -K\boldsymbol{x}$ back into the bracket of (M2.6):

$$V_{\text{new}}(\boldsymbol{x}) = \boldsymbol{x}^\top \Big[Q + K^\top R K + (A - BK)^\top P (A - BK) \Big] \boldsymbol{x}.$$

Expand $(A-BK)^\top P(A-BK) = A^\top PA - A^\top PBK - K^\top B^\top PA + K^\top B^\top PBK$ and add $K^\top RK$:

$$K^\top (R + B^\top PB) K = K^\top B^\top PA \quad \text{(by (M2.8))}$$
$$\Longrightarrow \; V_{\text{new}} = \boldsymbol{x}^\top [Q + A^\top PA - A^\top PBK]\boldsymbol{x} = \boldsymbol{x}^\top [Q + A^\top PA - A^\top PB(R + B^\top PB)^{-1} B^\top PA]\boldsymbol{x}.$$

This is (M2.10). Starting from the terminal cost P_0, each application gives the value matrix of a problem that is one step longer. If (A, B) is stabilisable and $Q \succ 0$, the sequence converges to the infinite-horizon matrix P_∞.

Toy example — scalar recursion, $a = 1.1$, $b = 0.5$, $q = r = 1$, $P_0 = 1$

$$P_{k+1} = 1 + 1.21 P_k - \frac{(0.55 P_k)^2}{1 + 0.25 P_k}$$

k	0	1	2	3	4	5	6	7	8	∞
P_k	1	1.968	2.596	2.905	3.036	3.089	3.109	3.117	3.120	3.1215

(35 iterations to 10^{-14}). For the drone, starting from $P_0 = Q$, the relative change falls below 10^{-12} after 236 iterations, and P_∞ is positive definite ($\lambda_{\min} = 3.27$).

In our drone project

Each iteration lets the controller "look one step further into the future". After a few hundred steps, which is 10 s of flight at 25 Hz, looking further no longer changes anything. The drone then plans as if the flight never ends, which is what an infinite-horizon LQR is.

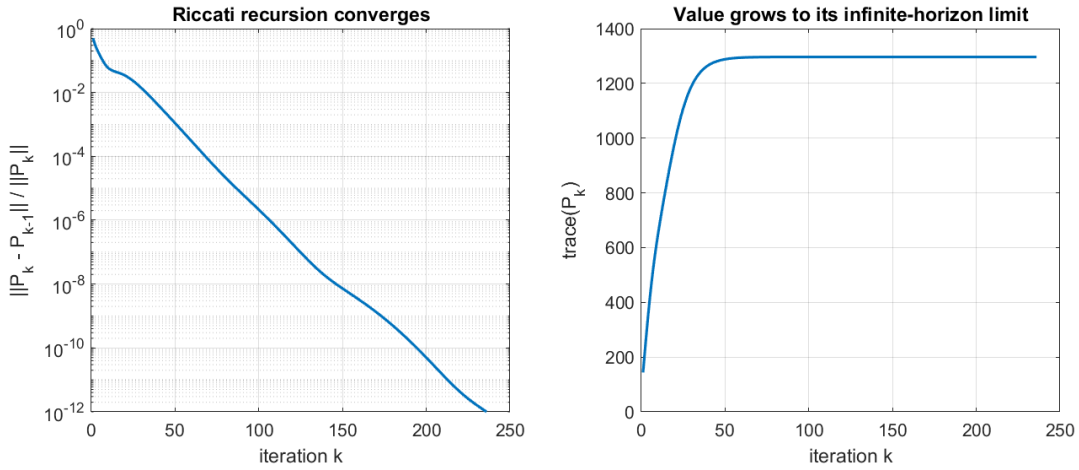


Figure 8: Toy model, Model 2 step 5: convergence of the Riccati recursion for the learned drone model.

Toy model: `step5_riccati_recursion.m`, `lib/riccati_recursion.m`.

5.6 Block 6 — DARE: Discrete Algebraic Riccati Equation

Equation M2.11 Steady-state Riccati equation

$$P = A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A + Q \tag{M2.11}$$

Symbol	Meaning	Dimension	Unit
P	stabilising solution, $P = P^\top \succ 0$	$\mathbb{R}^{12\times 12}$	as Q

Derivation

If $P_k \to P$, both sides of (M2.10) tend to the same limit, which gives (M2.11). Two further facts make P useful.

Optimal cost. Using the rearranged form of (M2.11), $P = Q + K^\top R K + (A - BK)^\top P (A - BK)$ (from the derivation of block 5), along the closed loop $\boldsymbol{x}_{k+1} = (A - BK)\boldsymbol{x}_k$:

$$\boldsymbol{x}_k^\top P \boldsymbol{x}_k - \boldsymbol{x}_{k+1}^\top P \boldsymbol{x}_{k+1} = \boldsymbol{x}_k^\top Q \boldsymbol{x}_k + \boldsymbol{u}_k^\top R \boldsymbol{u}_k.$$

Summing from $k = 0$ to ∞ (a telescoping sum, and $\boldsymbol{x}_k \to 0$) gives $J^* = \boldsymbol{x}_0^\top P \boldsymbol{x}_0$.

Direct solution. Instead of iterating, the optimality conditions $\boldsymbol{u}_k = -R^{-1}B^\top \boldsymbol{\lambda}_{k+1}$, $\boldsymbol{\lambda}_k = Q\boldsymbol{x}_k + A^\top \boldsymbol{\lambda}_{k+1}$ (co-state $\boldsymbol{\lambda}_k = P\boldsymbol{x}_k$) give the linear pencil

$$\begin{bmatrix} I & BR^{-1}B^\top \\ 0 & A^\top \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_{k+1} \\ \boldsymbol{\lambda}_{k+1} \end{bmatrix} = \begin{bmatrix} A & 0 \\ -Q & I \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_k \\ \boldsymbol{\lambda}_k \end{bmatrix}.$$

Its 12 stable generalised eigenvectors $[X_1; X_2]$ ($|\lambda| < 1$) span $\{\boldsymbol{\lambda} = P\boldsymbol{x}\}$, so $P = X_2 X_1^{-1}$. This is what `lib/dare_solve.m` implements, so no toolbox is needed.

Toy example — scalar DARE is a quadratic equation

$P = 1 + 1.21P - \dfrac{0.3025P^2}{1 + 0.25P}$. Multiplying by $(1 + 0.25P)$ and simplifying:

$$0.25P^2 - 0.46P - 1 = 0 \implies P = \frac{0.46 + \sqrt{0.46^2 + 1}}{0.5} = 3.1215$$

(the positive root), the limit of the recursion above. The eigenvector solver returns the same value.

Drone: DARE residual 4.2×10^{-14} (relative); agreement with the 236-step recursion 1.1×10^{-11}; closed-loop spectral radius $\rho(A - BK) = 0.950 < 1$, so it is stable. From $\boldsymbol{x}_0 = [0.5, -0.3, 0.2, 0, \dots]$: the simulated cost $\sum(\boldsymbol{x}^\top Q\boldsymbol{x} + \boldsymbol{u}^\top R\boldsymbol{u}) = 41.4373$ and the predicted $\boldsymbol{x}_0^\top P\boldsymbol{x}_0 = 41.4373$ agree.

In our drone project

The DARE gives the final P in one shot, and from it the gain K that we would upload to the drone. P is also reused: it becomes the terminal cost of the MPC (dashed arrow in fig. 2). That is the bridge between the two branches.

Toy model: `step6_dare.m`, `lib/dare_solve.m`.

5.7 Block 7 — Model Predictive Control (MPC)

Equation M2.12 Finite-horizon constrained optimal control problem

$$\begin{aligned} \min_{\boldsymbol{u}_0,\dots,\boldsymbol{u}_{N-1}} \quad & \sum_{k=0}^{N-1} (\boldsymbol{x}_k^\top Q\boldsymbol{x}_k + \boldsymbol{u}_k^\top R\boldsymbol{u}_k) + \boldsymbol{x}_N^\top P_f\, \boldsymbol{x}_N \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k, \quad \boldsymbol{x}_0 = \hat{\boldsymbol{x}}_k \text{ (estimated, block 11)}, \\ & \boldsymbol{u}_{\min} \le \boldsymbol{u}_k \le \boldsymbol{u}_{\max}, \qquad |C\boldsymbol{x}_k| \le \boldsymbol{x}_{\max} \quad (k = 1..N) \end{aligned} \tag{M2.12}$$

Symbol	Meaning	Dimension	Unit
N	prediction horizon	$\mathbb{N}$	steps ($25 = 1\,\text{s}$)
P_f	terminal weight (here $P_f = P$ from the DARE)	$\mathbb{R}^{12\times12}$	as Q
$\boldsymbol{u}_{\min}, \boldsymbol{u}_{\max}$	input limits (deviation)	$\mathbb{R}^4$	N, N m
C	selects the limited states (v_x, v_y, v_z, roll, pitch)	$\mathbb{R}^{5\times12}$	–
$\boldsymbol{x}_{\max}$	state limits	$\mathbb{R}^5$	m/s, rad

Derivation and reasoning

LQR solves (M2.5) without limits. Real drones have limits: thrust between roughly 5 and 25 N, torque that the rotors can actually produce, the `max_velocity` of 1 m/s, and a maximum tilt for a camera mount. Adding constraints destroys the closed-form solution, so MPC solves a *finite* version numerically, from the current measured state:

- the sum stops after N steps, which keeps the problem finite;
- the infinite tail $\sum_{k\ge N}$ is replaced by the terminal cost $\boldsymbol{x}_N^\top P_f \boldsymbol{x}_N$.

Why $P_f = P$ from the DARE. By section 5.6, $\boldsymbol{x}_N^\top P\boldsymbol{x}_N$ is *exactly* the optimal cost of the rest of the flight if no constraint is active after step N. With this choice, whenever the constraints are not active, the MPC solution coincides with the LQR solution. Block 8 verifies this numerically.

Toy example — a two-step problem

$a = 1.1$, $b = 0.5$, $q = r = 1$, $N = 2$, $P_f = 3.1215$, $x_0 = 1$, $|u_k| \le 0.5$:

$$\min_{u_0,u_1} \; x_0^2 + u_0^2 + x_1^2 + u_1^2 + 3.1215\,x_2^2 \quad \text{s.t. } x_{k+1} = 1.1x_k + 0.5u_k.$$

Drone: $N = 25$ (1 s look-ahead); $5 \le T \le 25$ N, i.e. $\delta T \in [-9.715, 10.285]$ N; $|\tau_{x,y}| \le 0.3$, $|\tau_z| \le 0.05$ N m; $|v_i| \le 1$ m/s; $|\theta_{x,y}| \le 0.35$ rad (20°).

In our drone project

MPC is a chess player. Every 40 ms it imagines the next second of flight, finds the best sequence of moves that never breaks a rule ("never faster than 1 m/s", "never ask a motor for more than it can give"), plays only the first move, and thinks again. For camera-based mapping, the speed and tilt limits protect image quality.

Toy model: `step7_mpc_problem.m`, `lib/mpc_build.m`.

5.8 Block 8 — Optimization formulation: Quadratic Program

Equations M2.13 – M2.15 Stacked prediction, cost and constraints

$$\mathbf{X} = S_x \boldsymbol{x}_0 + S_u \mathbf{U}, \qquad \mathbf{X} = \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_N \end{bmatrix}, \tag{10}$$

$$\mathbf{U} = \begin{bmatrix} \boldsymbol{u}_0 \\ \vdots \\ \boldsymbol{u}_{N-1} \end{bmatrix}, \tag{11}$$

$$S_x = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}, \tag{12}$$

$$S_u = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ A^{N-1}B & \cdots & AB & B \end{bmatrix} \tag{M2.13}$$

$$\min_{\mathbf{U}} \; \tfrac{1}{2}\mathbf{U}^\top H \mathbf{U} + \boldsymbol{f}^\top \mathbf{U}, \quad H = 2(S_u^\top \bar{Q} S_u + \bar{R}), \quad \boldsymbol{f} = 2 S_u^\top \bar{Q} S_x \boldsymbol{x}_0 \tag{M2.14}$$

$$\text{s.t.} \quad G\mathbf{U} \le \boldsymbol{h}, \qquad A_{eq}\mathbf{U} = \boldsymbol{b}_{eq} \tag{M2.15}$$

with $\bar{Q} = \mathrm{diag}(Q, \ldots, Q, P_f)$ and $\bar{R} = \mathrm{diag}(R, \ldots, R)$.

Symbol	Meaning	Dimension	Unit
$\mathbf{X}$, $\mathbf{U}$	stacked predicted states / inputs	$\mathbb{R}^{12N}$, $\mathbb{R}^{4N}$	mixed
S_x, S_u	prediction matrices	$\mathbb{R}^{12N\times 12}$, $\mathbb{R}^{12N\times 4N}$	–
$\bar{Q}$, $\bar{R}$	block-diagonal weights	$\mathbb{R}^{12N\times 12N}$, $\mathbb{R}^{4N\times 4N}$	–
H, $\boldsymbol{f}$	QP Hessian and linear term	$\mathbb{R}^{4N\times 4N}$, $\mathbb{R}^{4N}$	–
G, $\boldsymbol{h}$	inequality constraints	$\mathbb{R}^{n_c\times 4N}$, $\mathbb{R}^{n_c}$	mixed
A_{eq}, $\boldsymbol{b}_{eq}$	equality constraints (optional)	$\mathbb{R}^{n_e\times 4N}$, $\mathbb{R}^{n_e}$	mixed

Derivation

(M2.13) Prediction. Apply $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ repeatedly: $\boldsymbol{x}_1 = A\boldsymbol{x}_0 + B\boldsymbol{u}_0$, $\boldsymbol{x}_2 = A^2\boldsymbol{x}_0 + AB\boldsymbol{u}_0 + B\boldsymbol{u}_1$, and in general

$$\boldsymbol{x}_k = A^k\boldsymbol{x}_0 + \sum_{j=0}^{k-1} A^{k-1-j}B\boldsymbol{u}_j.$$

Stacking $k = 1..N$ gives (M2.13). The future states are *linear* in the unknown inputs.

(M2.14) Cost. The cost of (M2.12) is $\boldsymbol{x}_0^\top Q\boldsymbol{x}_0 + \mathbf{X}^\top\bar{Q}\mathbf{X} + \mathbf{U}^\top\bar{R}\mathbf{U}$. Substitute $\mathbf{X}$:

$$\mathbf{U}^\top(S_u^\top\bar{Q}S_u + \bar{R})\mathbf{U} + 2\,\boldsymbol{x}_0^\top S_x^\top\bar{Q}S_u\,\mathbf{U} + \underbrace{\boldsymbol{x}_0^\top(Q + S_x^\top\bar{Q}S_x)\boldsymbol{x}_0}_{\text{does not depend on }\mathbf{U}}.$$

Dropping the constant and writing it as $\frac{1}{2}\mathbf{U}^\top H\mathbf{U} + \boldsymbol{f}^\top\mathbf{U}$ gives H and $\boldsymbol{f}$ in (M2.14). $H \succ 0$ because $\bar{R} \succ 0$, so the problem is *convex*, with a single global minimum and no local traps.

(M2.15) Constraints. Input limits: $[I; -I]\,\mathbf{U} \le [\mathbf{U}_{\max}; -\mathbf{U}_{\min}]$. State limits $|\bar{C}\mathbf{X}| \le \bar{\boldsymbol{x}}_{\max}$ become $\bar{C}S_u\mathbf{U} \le \bar{\boldsymbol{x}}_{\max} - \bar{C}S_x\boldsymbol{x}_0$ and $-\bar{C}S_u\mathbf{U} \le \bar{\boldsymbol{x}}_{\max} + \bar{C}S_x\boldsymbol{x}_0$. Stacking everything gives $G\mathbf{U} \le \boldsymbol{h}(\boldsymbol{x}_0)$. Equality constraints appear, for example, as a terminal condition $\boldsymbol{x}_N = 0$, i.e. (last block row of S_u)$\,\mathbf{U} = -A^N\boldsymbol{x}_0$, or in the "sparse" formulation, where both $\mathbf{X}$ and $\mathbf{U}$ are unknowns and the dynamics themselves are the equalities.

Solver. The toy model solves the QP with a primal–dual interior-point method (`lib/qp_solve.m`). It applies Newton steps on the KKT conditions $H\mathbf{U} + \boldsymbol{f} + G^\top\boldsymbol{z} + A_{eq}^\top\boldsymbol{y} = 0$, $G\mathbf{U} + \boldsymbol{s} = \boldsymbol{h}$, $s_i z_i = 0$, keeping $\boldsymbol{s}, \boldsymbol{z} > 0$. It replaces `quadprog`, so no Optimization Toolbox is needed.

Toy example — the two-step problem of block 7, solved by hand

$$S_x = \begin{bmatrix} 1.1 \\ 1.21 \end{bmatrix}, \quad S_u = \begin{bmatrix} 0.5 & 0 \\ 0.55 & 0.5 \end{bmatrix}, \quad \bar{Q} = \mathrm{diag}(1,\, 3.1215),$$

$$H = 2\left(\begin{bmatrix} 1.1942 & 0.8584 \\ 0.8584 & 0.7804 \end{bmatrix} + I\right) = \begin{bmatrix} 4.3885 & 1.7168 \\ 1.7168 & 3.5607 \end{bmatrix}, \qquad \boldsymbol{f} = \begin{bmatrix} 5.2547 \\ 3.7770 \end{bmatrix}.$$

- **No limits:** $\mathbf{U}^* = -H^{-1}\boldsymbol{f} = [-0.9643,\ -0.5958]$. The first move equals the LQR move $-Kx_0 = -0.9643$ *exactly*, as promised by $P_f = P$.
- **With $|u| \le 0.5$:** $\mathbf{U}^* = [-0.5,\ -0.5]$. Both limits are active and the KKT multipliers are positive.
- **With $A_{eq}\mathbf{U} = b_{eq}$ forcing $x_2 = 0$** ($[0.55\ \ 0.5]\,\mathbf{U} = -1.21$): $\mathbf{U}^* = [-1.3057,\ -0.9837]$. Check: $0.55(-1.3057) + 0.5(-0.9837) = -1.21$.

Drone ($N = 25$): 100 decision variables and 450 inequality constraints. For a 5 cm offset (no limit active) the MPC input equals $-K\boldsymbol{x}_0$ to all printed digits. For a 2 m offset the unconstrained LQR would demand 1.39 times the allowed input; the QP saturates the thrust at +10.285 N and plans a velocity that touches but never exceeds 1 m/s. Solved in 10 interior-point iterations (about 8 ms).

In our drone project

Here the physics disappears into a few matrices. To the solver, "fly the drone well without breaking the rules" is just "find the lowest point of a bowl (H, $\boldsymbol{f}$) inside a fence (G, $\boldsymbol{h}$)". Because the bowl has one bottom, the answer is unique and can be found reliably in milliseconds. That is what makes MPC feasible at 25 Hz.

Toy model: `step8_qp_formulation.m`, `lib/mpc_build.m`, `lib/qp_solve.m`.

5.9 Block 9 — MPC control input (receding horizon)

Equation M2.16 Receding-horizon control law

$$\mathbf{U}^*(\boldsymbol{x}_k) = [\boldsymbol{u}_0^*,\ \boldsymbol{u}_1^*,\ \ldots,\ \boldsymbol{u}_{N-1}^*] = \arg\min \mathrm{QP}(\boldsymbol{x}_k) \quad \Longrightarrow \quad \boldsymbol{u}_k = \boldsymbol{u}_0^*(\boldsymbol{x}_k), \quad k \leftarrow k+1 \ \text{(M2.16)}$$

Symbol	Meaning	Dimension	Unit
$\mathbf{U}^*(\boldsymbol{x}_k)$	optimal plan computed at time k	$\mathbb{R}^{4N}$	N, N m
$\boldsymbol{u}_0^*$	first element of the plan, the only one applied	$\mathbb{R}^4$	N, N m

Derivation and reasoning

The plan $\mathbf{U}^*$ is optimal *for the model*. The model is never perfect (it was learned, and wind is not in it). Re-solving at every sample from the newly measured state $\boldsymbol{x}_{k+1}$ turns the open-loop plan into a feedback law $\boldsymbol{u}_k = \kappa(\boldsymbol{x}_k)$: errors are corrected 25 times per second instead of accumulating. Applying $\boldsymbol{u}_0^*$ and shifting the horizon forward by one step is the *receding horizon* principle.

Toy example — 8 s on the learned model

Start 1.5 m, 1.0 m and 1.2 m away from the setpoint. At $k = 0$ the plan begins with $\boldsymbol{u}_0^* = [10.285, -0.279, -0.300, 0.002]$: full upward thrust and the maximum allowed torque. Over the run, MPC respects $|v_i| \le 1$ m/s (maximum 1.000), while LQR clipped to the same input limits reaches 1.172 m/s. MPC converges to $\|\boldsymbol{x}\| = 1.3 \times 10^{-5}$. The state limits never had to be relaxed. The mean solve time is 2.2 ms against a 40 ms budget.

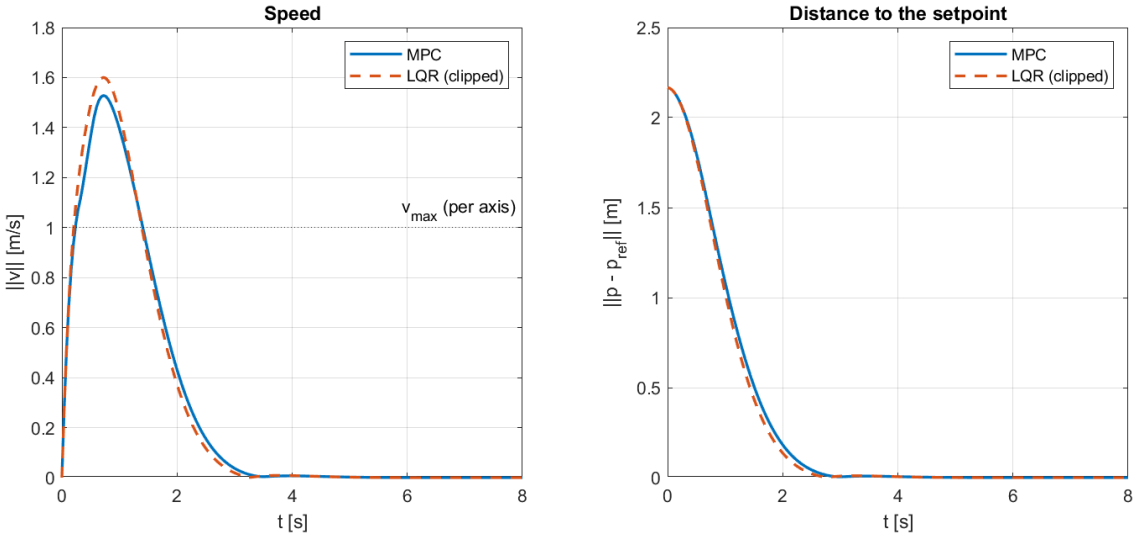


Figure 9: Toy model, Model 2 step 9: receding-horizon MPC versus input-clipped LQR on the learned model.

In our drone project

"Plan one second ahead, act for 40 milliseconds, look again." Because the plan is recomputed from fresh EKF estimates, the MPC inherits the robustness of feedback while still respecting every limit.

Toy model: `step9_mpc_control_input.m`, `lib/mpc_solve.m`.

5.10 Block 10 — Apply to the drone

Equations M2.17 – M2.18 From optimal input to motor speeds

$$\boldsymbol{u}_k^{\text{abs}} = \boldsymbol{u}_{\text{trim}} + \begin{cases} \boldsymbol{u}_0^*(\boldsymbol{x}_k) & \text{(MPC)} \\ -K\boldsymbol{x}_k & \text{(LQR)} \end{cases} = \begin{bmatrix} T_k \\ \boldsymbol{\tau}_k \end{bmatrix}, \qquad \boldsymbol{u}_{\text{trim}} = \begin{bmatrix} mg_0 \\ \mathbf{0} \end{bmatrix} \tag{M2.17}$$

$$\boldsymbol{f} = \Gamma^{-1}\boldsymbol{u}_k^{\text{abs}}, \qquad \Omega_i = \sqrt{f_i/k_f} \quad \text{(as in M1.24–M1.25)} \tag{M2.18}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{u}_{\text{trim}}$	hover input (operating point)	$\mathbb{R}^4$	N, N m
$\boldsymbol{u}_k^{\text{abs}}$	absolute thrust and torque	$\mathbb{R}^4$	N, N m
$\boldsymbol{\Omega}$	motor speeds	$\mathbb{R}^4$	rad/s

Derivation

Models and controllers of Model 2 work in deviations (M2.1). The real motors need absolute values, so the hover trim is added back. The rest is the same allocation as Model 1 (section 4.8). The loop is closed by block 11, which estimates the new state of the *nonlinear* drone from its sensors and hands it to block 9 (MPC) or to the LQR law. Logged data can also be fed back to block 1 to re-identify the model (outer dashed loop of fig. 2). In the toy run of this block the controllers are still given the *true* state plus a little noise, so that blocks 1–10 can be checked on their own. Block 11 removes that shortcut.

Toy example — one control sample, then a whole flight

Drone at $[1.5, -1.0, 0.8]$ m, at rest; target $[0, 0, 2]$. The MPC gives $\delta\boldsymbol{u} = [10.285, -0.279, -0.300, 0.002]$, so $\boldsymbol{u} = [25.000, -0.279, -0.300, 0.002]$, $\boldsymbol{f} = [7.115, 5.327, 6.539, 6.019]$ N and $\boldsymbol{\Omega} = [912, 789, 875, 839]$ rad/s.
Nonlinear flight, 10 s with gusts and sensor noise, controllers designed only on the DMDc model:

controller	max $\lvert v_i \rvert$ [m/s]	error at 10 s [m]	motor speeds [rad/s]
MPC	0.994	0.015	584 – 913
LQR (clipped)	1.118	0.031	613 – 911

Both stabilise the real drone. Only MPC honours `max_velocity` = 1 m/s.

In our drone project

This is the moment of truth: a controller designed on a learned, linear, approximate model flies the real, nonlinear, gusty, motor-limited drone. In the project this block is the same motor mixer as Model 1, fed through ArduPilot.

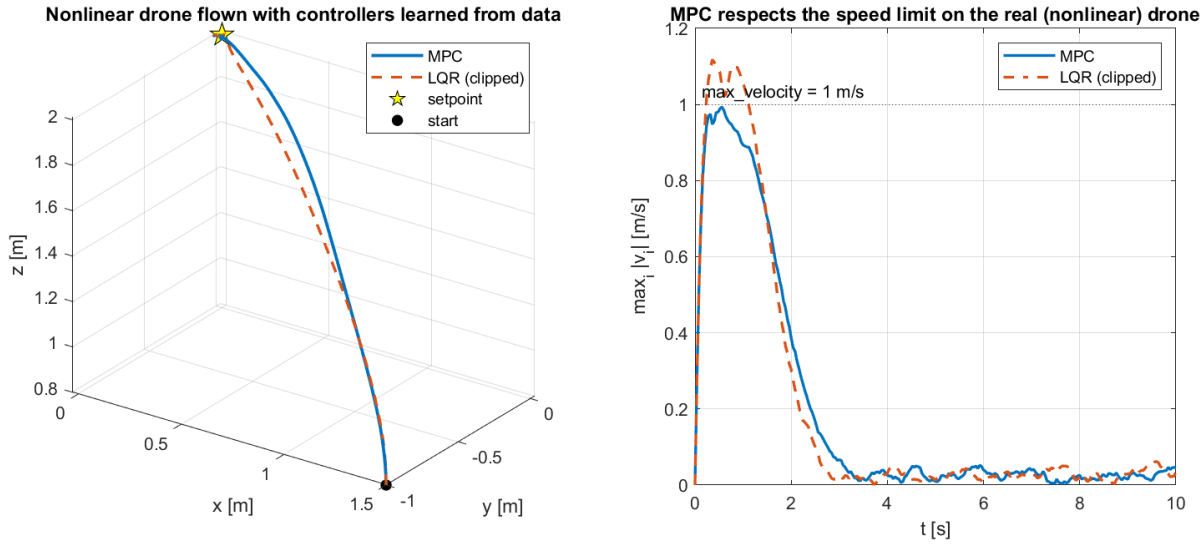


Figure 10: Toy model, Model 2 step 10: the nonlinear drone flown by controllers that were designed on a model learned from data.

Toy model: `step10_apply_to_drone.m`, `lib/motor_allocation.m`.

5.11 Block 11 — State estimation: Extended Kalman Filter

Why this block exists. Blocks 9 and 10 need the full 12-number state $\boldsymbol{x}_k$ every 40 ms. No sensor gives it. The camera gives *pixels*, the gyroscope gives *body rates*, and nothing measures the velocity at all. Raw pixels fed straight into $-K\boldsymbol{x}$ would make no sense, and raw noise multiplied by the large gains of K would shake the motors.
Where it sits. It closes Model 2's loop: apply to drone (10) → the drone moves → camera and gyro → **EKF (11)** → $\hat{\boldsymbol{x}}_k$ → MPC (9, as $\boldsymbol{x}_0$) or LQR ($-K\hat{\boldsymbol{x}}_k$) → apply (10). In fig. 2 it replaces the earlier arrow "measured $\boldsymbol{x}_{k+1}$".
What makes it Model 2. It *predicts with the learned model* A, B (no mass, inertia or rotor constants needed), the same model the controllers trust. It learns its own noise level from the DMDc residuals. It is *extended* because the camera equation is nonlinear (division by depth, rotation), so it is linearized with a Jacobian at every step.

Two new symbols. Model 2 already uses P (value matrix), Q (state weight) and K (LQR gain). So the filter's covariance is written Σ, its process noise Q_e, and its gain L ("estimator gain"). The maths is exactly that of Model 1, block 4.

Equations M2.19 – M2.24 Extended Kalman filter on the learned model

Prediction (learned model)

$$\hat{\boldsymbol{x}}_{k+1} = A\,\hat{\boldsymbol{x}}_k + B\,\boldsymbol{u}_k, \qquad \Sigma_{k+1} = A\,\Sigma_k A^\top + Q_e \tag{M2.19}$$

$$Q_e = \mathrm{cov}(\boldsymbol{x}_{k+1} - A\boldsymbol{x}_k - B\boldsymbol{u}_k) \quad \text{(on a flight not used for learning)} \tag{M2.20}$$

Measurement models (camera marker j, gyroscope)

$$\boldsymbol{q}_j = R_{bc}^\top R(\boldsymbol{\theta})^\top(\boldsymbol{\ell}_j - \boldsymbol{p}), \qquad \boldsymbol{z}_j = \begin{bmatrix} f\,q_1/q_3 + c_x \\ f\,q_2/q_3 + c_y \end{bmatrix}, \qquad \boldsymbol{z}_\omega = \boldsymbol{\omega} \tag{M2.21}$$

$$H_j = \frac{f}{q_3}\begin{bmatrix} 1 & 0 & -q_1/q_3 \\ 0 & 1 & -q_2/q_3 \end{bmatrix}\Big[\, -R_{bc}^\top R^\top \quad 0 \quad R_{bc}^\top[R^\top(\boldsymbol{\ell}_j - \boldsymbol{p})]_\times J_r(\boldsymbol{\theta}) \quad 0 \,\Big], \qquad H_\omega = [\,0\;\;0\;\;0\;\;I\,] \tag{M2.22}$$

Correction

$$L = \Sigma H^\top \left(H\Sigma H^\top + R_n\right)^{-1} \tag{M2.23}$$

$$\hat{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}} + L\big(\boldsymbol{z} - h(\hat{\boldsymbol{x}})\big), \qquad \Sigma \leftarrow (I - LH)\,\Sigma\,(I - LH)^\top + LR_nL^\top \tag{M2.24}$$

Symbol	Meaning	Dimension	Unit
$\hat{\boldsymbol{x}}_k$	estimate of the deviation state $[\delta\boldsymbol{p}; \boldsymbol{v}; \boldsymbol{\theta}; \boldsymbol{\omega}]$	$\mathbb{R}^{12}$	m, m/s, rad, rad/s
Σ_k	covariance of the estimation error	$\mathbb{R}^{12\times12}$	m^2, $(\mathrm{m/s})^2$, …
Q_e	process noise: per-step error of the learned model	$\mathbb{R}^{12\times12}$	as Σ
$\boldsymbol{\ell}_j$	known position of floor marker j (world)	$\mathbb{R}^3$	m
$\boldsymbol{p},\ \boldsymbol{\theta}$	drone position $\boldsymbol{p}_{\mathrm{ref}} + \delta\boldsymbol{p}$ and attitude $\mathrm{Log}(R)$	$\mathbb{R}^3$	m, rad
R_{bc}	camera axes written in the body frame, $\mathrm{diag}(1,-1,-1)$	SO(3)	–
$\boldsymbol{q}_j$	marker j in camera axes (q_3 = depth)	$\mathbb{R}^3$	m
$f,\ c_x,\ c_y$	focal length and image centre	$\mathbb{R}$	px (400, 320, 240)
$\boldsymbol{z}_j,\ \boldsymbol{z}_\omega$	pixel of marker j; gyroscope reading	$\mathbb{R}^2$, $\mathbb{R}^3$	px, rad/s
$J_r(\boldsymbol{\theta})$	right Jacobian of SO(3)	$\mathbb{R}^{3\times3}$	–
H	stacked Jacobian of all visible markers and the gyro	$\mathbb{R}^{m\times12}$	px/m, px/rad, –
R_n	measurement noise, $\mathrm{diag}(1\,\mathrm{px}^2, \dots, 0.01^2, \dots)$	$\mathbb{R}^{m\times m}$	px^2, $(\mathrm{rad/s})^2$
L	Kalman (estimator) gain	$\mathbb{R}^{12\times m}$	mixed

Derivation

(M2.19) Prediction. The learned model is already linear. If the estimation error $\boldsymbol{e}_k = \boldsymbol{x}_k - \hat{\boldsymbol{x}}_k$ has mean zero, then $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k + \boldsymbol{w}_k$ (with $\boldsymbol{w}_k$ the model's error) gives $\boldsymbol{e}_{k+1} = A\boldsymbol{e}_k + \boldsymbol{w}_k$. Taking the mean gives the first equation. Taking $\mathbb{E}[\boldsymbol{e}_{k+1}\boldsymbol{e}_{k+1}^\top]$ gives $A\Sigma_kA^\top + Q_e$, exactly as in (M1.11), with $F = A$. No linearization is needed in this half.

(M2.20) Q_e from data. The residual $\boldsymbol{w}_k = \boldsymbol{x}_{k+1} - A\boldsymbol{x}_k - B\boldsymbol{u}_k$ *is* the per-step error of the learned model, so its covariance is the natural Q_e. It must be computed on a flight that was *not* used for DMDc, because least squares made the training residuals as small as possible, which would make the filter over-confident. In the toy model the test flight has gusts twice as strong as the recorded flights. Variance grows with the square of the amplitude, so Q_e is multiplied by 4.

(M2.21) Pinhole camera. The marker seen from the drone is the world vector $\boldsymbol{\ell}_j - \boldsymbol{p}$. Since R turns body vectors into world vectors, $R^\top$ turns it into body axes, and $R_{bc}^\top$ into camera axes (forward, right, down). By similar triangles, a point at sideways offset q_1 and depth q_3 lands $f\,q_1/q_3$ pixels from the image centre. The division by q_3, and $R(\boldsymbol{\theta})$ inside, make the measurement *nonlinear*. This is why we need an *extended* filter. The gyroscope reads $\boldsymbol{\omega}$ directly, which is linear.

(M2.22) Jacobian by the chain rule. pixel $\leftarrow \boldsymbol{q} \leftarrow$ state.

- $\partial\boldsymbol{z}_j/\partial\boldsymbol{q} = \frac{f}{q_3}\begin{bmatrix}1 & 0 & -q_1/q_3\\ 0 & 1 & -q_2/q_3\end{bmatrix}$ (quotient rule).
- Moving the drone moves the marker the other way: $\partial\boldsymbol{q}/\partial\boldsymbol{p} = -R_{bc}^\top R^\top$.
- Turning the drone: with the right Jacobian, $R(\boldsymbol{\theta}+\boldsymbol{d}) \approx R(\boldsymbol{\theta})\mathrm{Exp}(J_r\boldsymbol{d})$, so $R(\boldsymbol{\theta}+\boldsymbol{d})^\top\boldsymbol{a} \approx (I - [J_r\boldsymbol{d}]_\times)R^\top\boldsymbol{a} = R^\top\boldsymbol{a} + [R^\top\boldsymbol{a}]_\times J_r\boldsymbol{d}$, giving $\partial\boldsymbol{q}/\partial\boldsymbol{\theta} = R_{bc}^\top[R^\top(\boldsymbol{\ell}_j - \boldsymbol{p})]_\times J_r(\boldsymbol{\theta})$, where $J_r(\boldsymbol{\theta}) = I - \frac{1-\cos\phi}{\phi^2}[\boldsymbol{\theta}]_\times + \frac{\phi-\sin\phi}{\phi^3}[\boldsymbol{\theta}]_\times^2$, $\phi = \|\boldsymbol{\theta}\|$ ($\approx I$ near hover).
- Velocity and body rate do not move pixels, so those columns are zero.

The toy model checks this against a numerical derivative: the largest difference is 1.2×10^{-7} px.

(M2.23)–(M2.24) Correction. Word for word the derivation of (M1.12)–(M1.14), with $K \to L$ and $P \to \Sigma$. The 12-state of Model 2 is a plain vector (the attitude is already $\boldsymbol{\theta} = \mathrm{Log}R$), so the

correction is an ordinary addition. With up to 70 measurements per step, the Joseph form (the second equation, equal to $(I - LH)\Sigma$ for the optimal L) is used because it keeps Σ symmetric and positive in floating point.

	EKF of Model 1 (block 4)	**EKF of Model 2 (block 11)**
Prediction model	nonlinear physics, RK4, needs m, J	learned linear A, B, no parameters
Where it is "extended"	prediction ($F = I + A\Delta t$ from Jacobians)	measurement (pinhole camera)
State	18 numbers on $\mathbb{R}^9 \times \mathrm{SO}(3)$, error-state	plain 12-vector of deviations
Camera measurement	a ready-made pose from visual odometry	raw pixels of known markers
Noise Q	chosen from the gust model	learned from DMDc residuals

Toy example — altitude from one pixel, by hand

A camera looks down from height h at a marker $d = 1\,\mathrm{m}$ to the side: $u = fd/h$, a curve, not a line. Prior $\hat{h} = 2\,\mathrm{m}$, $\Sigma = 0.04\,\mathrm{m}^2$ ($\sigma = 20\,\mathrm{cm}$), $f = 400\,\mathrm{px}$; measured $u = 210\,\mathrm{px}$ with $R_n = 4\,\mathrm{px}^2$.

$$u_{\mathrm{pred}} = \frac{400 \cdot 1}{2} = 200, \quad H = -\frac{fd}{\hat{h}^2} = -100\ \frac{\mathrm{px}}{\mathrm{m}}, \quad L = \frac{\Sigma H}{H^2\Sigma + R_n} = \frac{-4}{404} = -0.0099,$$

$$\hat{h} \leftarrow 2 + (-0.0099)(210 - 200) = 1.901\ \mathrm{m}, \qquad \Sigma \leftarrow (1 - LH)\Sigma = 0.0004\ \mathrm{m}^2\ (\sigma = 2\ \mathrm{cm}).$$

Inverting the pixel exactly gives $400/210 = 1.905\,\mathrm{m}$. The 4 mm difference is the price of linearizing the curve.

The drone (toy model step 11: 10 s nonlinear flight, gusts, 1 px pixel noise, 0.01 rad/s gyro noise, initial estimate 20 cm wrong; up to 34 markers visible):

controller fed by	pos. error	vel. error	max $\lvert v_i \rvert$	error at 10 s	in $\pm 3\sigma$
MPC + EKF	3.4 mm	2.9 cm/s	1.060 m/s	1.5 cm	98.7%
LQR + EKF	3.3 mm	3.4 cm/s	1.221 m/s	3.5 cm	96.9%
MPC, true state (as in block 10)	–	–	1.010 m/s	0.5 cm	–

The velocity is never measured, yet it is known to 3 cm/s. The normalized innovation averages 0.95 (ideal: 1), so the filter is *consistent*: it knows how uncertain it is. The first ≈ 0.5 s show the filter converging from its wrong start.

Honest note: on the estimate, MPC overshoots the 1 m/s limit by 6%. It enforces the limit on the *estimated* and predicted state, and gusts plus estimation error make the truth a little different. The standard fix is constraint tightening: plan with a small margin, e.g. 0.95 m/s.

In our drone project — the "VIO" of Model 2

This block is what lets Model 2 fly on our own monocular camera and IMU, which is the title of our project. In the toy model the camera sees markers at known positions (think printed AprilTags on the floor). A full VIO system tracks *natural* image features whose positions are unknown and adds them to the state (that is SLAM, and how the 3D map is built), but the predict/correct structure and the Jacobian (M2.22) stay the same. In today's SITL set-up, ArduPilot's EKF3 publishes `/ap/v1/pose/filtered`. Block 11 is the version that uses the learned model instead of ArduPilot's built-in physics.

Toy model: `step11_state_estimation_ekf.m`, `lib/camera_model.m`, `lib/camera_measure.m`, `lib/so3_right_jacobian.m`, `lib/ekf2_predict.m`, `lib/ekf2_update.m`; `toy_model3/M2_4_EKF_State_Estimation.mlx`.

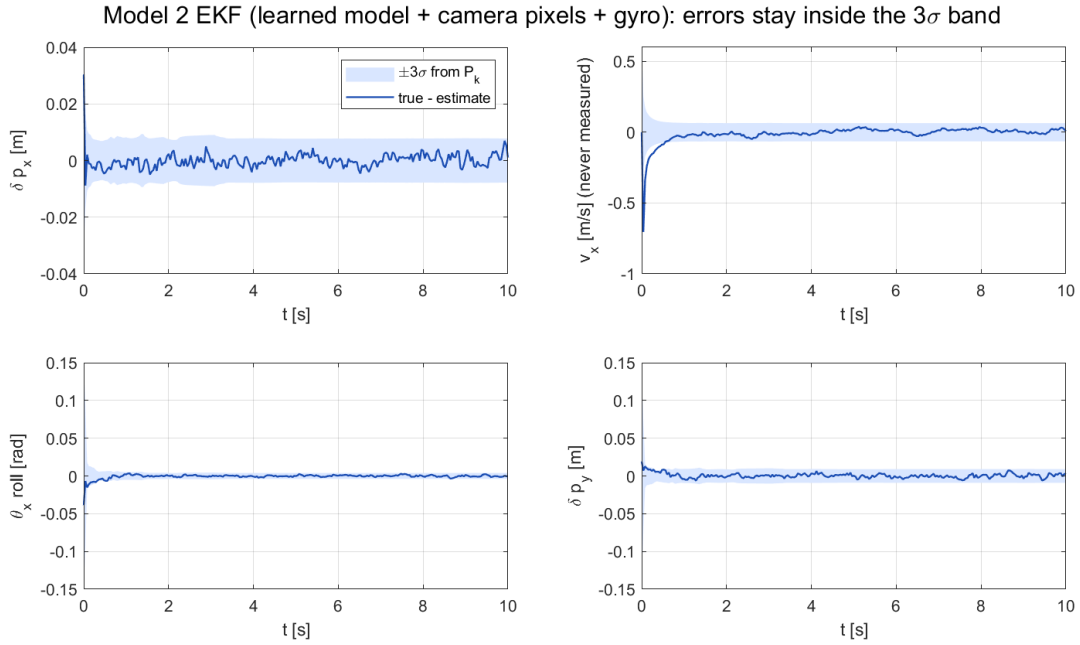


Figure 11: Toy model, Model 2 step 11: estimation errors stay inside the $\pm 3\sigma$ band predicted by Σ_k, including the never-measured velocity.

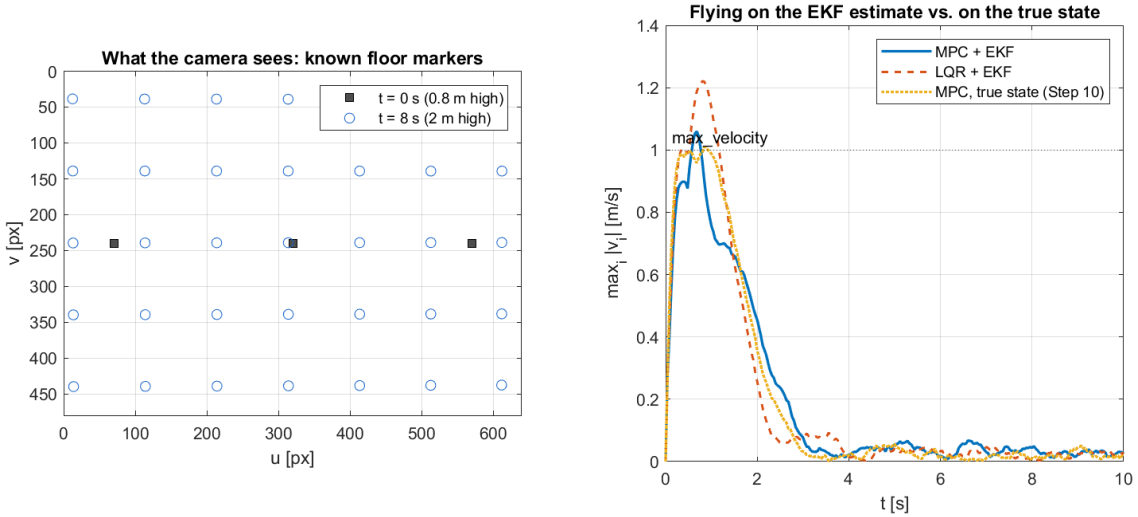


Figure 12: Toy model, Model 2 step 11. Left: what the downward camera sees at 0.8 m and at 2 m. Right: flying on the EKF estimate compared with flying on the true state.

5.12 Recap of Model 2

#	Block	In	Out	The idea in one sentence
1	Flight data	drone, excitation	$\{\boldsymbol{x}_k\}, \{\boldsymbol{u}_k\}$	Record what the drone did and what we told it, as deviations from hover.
2	DMDc	X, X^+, U	A, B	Least squares finds the linear rule that best predicts the next sample.
3	Bellman	A, B, Q, R	$V(\boldsymbol{x}) = \boldsymbol{x}^\top P \boldsymbol{x}$	The best plan from here is the best first move plus the best plan from where it leads.
4	LQR	A, B, R, P	K	Minimising a quadratic gives a linear law $\boldsymbol{u} = -K\boldsymbol{x}$.
5	Riccati recursion	P_0	P_∞	Look one more step ahead each iteration until nothing changes.
6	DARE	A, B, Q, R	P, final K	Jump straight to that steady state; $\boldsymbol{x}_0^\top P \boldsymbol{x}_0$ is the optimal cost.
7	MPC	model, limits, P_f	optimisation problem	Plan the next second optimally without breaking any rule.
8	QP	problem, $\boldsymbol{x}_0$	$H, \boldsymbol{f}, G, \boldsymbol{h}$, $\mathbf{U}^*$	Substitute the predictions: a convex bowl inside a fence.
9	MPC input	$\hat{\boldsymbol{x}}_k$	$\boldsymbol{u}_0^*$	Apply only the first move, then replan with fresh data.
10	Apply to drone	$\boldsymbol{u}_0^*$ or $-K\hat{\boldsymbol{x}}_k$	$\Omega_1..\Omega_4$	Add hover thrust back, split among rotors, fly the real drone.
11	EKF	$\boldsymbol{u}_k$, pixels, gyro, A, B	$\hat{\boldsymbol{x}}_k$, Σ_k	Predict with the learned model, correct with what the camera sees; the controllers fly on the result.

6 How the Two Models Fit Together in Our Project

6.1 One drone, two ways of thinking

	Model 1 (model-based)	Model 2 (data-driven)
Where the model comes from	Newton/Euler physics, needs m, J, rotor constants	Logged flight data (DMDc), no parameters needed
Model type	Nonlinear, continuous time, attitude on SO(3)	Linear, discrete time, around hover
Estimation	Error-state EKF, physics prediction, camera pose + IMU	EKF on the learned model, camera pixels + gyro (block 11)
Controller	Geometric SO(3) cascade (position → attitude)	LQR (gain table) or MPC (online QP)
Valid range	Any attitude except upside-down	Near hover (small tilts)
Limits / constraints	Only by saturation	Explicit in MPC (`max_velocity`, tilt, thrust)
On-board cost per step	A few 3×3 products	LQR: one 4×12 product; MPC: a QP (about 2 ms)
Code in the repository	`so3_controller.py`, `state_adapter.py`	toy model `model2_dmdc_lqr_mpc/`

6.2 The complete picture

1. The monocular camera produces images, visual odometry turns them into a pose, and the IMU produces rates.
2. An EKF fuses them with a model into one state estimate $\hat{\boldsymbol{x}}$. In Model 1 the model is the physics (M1.10–M1.14, with M1.1–M1.9). In Model 2 it is the learned A, B, corrected with raw camera

pixels (M2.19–M2.24). This is the "VIO" of our title. The same state, with the camera poses, is what the 3D map is built from.

3. Either the **geometric controller** (M1.15–M1.23) or a **data-driven LQR/MPC** (M2.5–M2.16) turns $\hat{\boldsymbol{x}}$ and the mission waypoints into thrust and torques.
4. The **allocation** (M1.24–M1.25 = M2.17–M2.18) turns them into four motor speeds, and the drone moves.
5. Logged flights can be fed back into DMDc to keep Model 2's A, B up to date.

6.3 The toy model: `toy_model2`

Every block of both flows is a MATLAB script that prints its **inputs** and **outputs** (name, size, unit, meaning, value). It first reproduces the hand-worked toy example of this document (Part A) and then passes real data to the next step (Part B). Results are saved to `results/*.mat` and figures to `figures/`. No toolbox is required: a QP solver and a DARE solver are included.

Folder / script	Equations	Inputs → outputs
`toy_model2/model1_nonlinear_ekf_so3/`	(run `run_all_model1`)	
`step1_nonlinear_dynamics`	M1.1–4	$\boldsymbol{x}, \boldsymbol{u} \to \dot{\boldsymbol{x}}$; open-loop flight
`step2_taylor_jacobian`	M1.5–6	$(\hat{\boldsymbol{x}}, \hat{\boldsymbol{u}}) \to A, B$; numerical check
`step3_local_error_dynamics`	M1.7–9	$A, B, \Delta t \to F$; linear vs. true error
`step4_error_state_ekf`	M1.10–14	$\boldsymbol{u}_k, \boldsymbol{z}_k \to \hat{\boldsymbol{x}}_k, P_k$
`step5_position_control`	M1.15–16	$\hat{\boldsymbol{p}}, \hat{\boldsymbol{v}}, \boldsymbol{p}_d, \boldsymbol{v}_d, \ddot{\boldsymbol{p}}_d \to \boldsymbol{e}_p, \boldsymbol{e}_v, \boldsymbol{a}_d$
`step6_force_attitude`	M1.17–19	$\boldsymbol{a}_d, \psi_d \to \boldsymbol{F}_d, \boldsymbol{b}_{3d}, R_d$
`step7_attitude_control`	M1.20–23	$\hat{R}, \hat{\boldsymbol{\omega}}, R_d, \boldsymbol{F}_d \to \boldsymbol{e}_R, \boldsymbol{e}_\omega, \boldsymbol{\tau}, T$
`step8_actuators_feedback`	M1.24–25	$T, \boldsymbol{\tau} \to \boldsymbol{\Omega}$; full closed loop
`toy_model2/model2_dmdc_lqr_mpc/`	(run `run_all_model2`)	
`step1_flight_data`	M2.1	drone + excitation → $\{\boldsymbol{x}_k\}, \{\boldsymbol{u}_k\}$
`step2_dmdc`	M2.2–4	$X, X^+, U \to A, B$
`step3_bellman_cost`	M2.5–7	$A, B \to Q, R$; Bellman check
`step4_lqr_gain`	M2.8–9	$A, B, R, P \to K$
`step5_riccati_recursion`	M2.10	$P_0 \to P_\infty$
`step6_dare`	M2.11	$A, B, Q, R \to P, K$
`step7_mpc_problem`	M2.12	limits, N, P_f → MPC problem
`step8_qp_formulation`	M2.13–15	problem, $\boldsymbol{x}_0 \to H, \boldsymbol{f}, G, \boldsymbol{h}, \mathbf{U}^*$
`step9_mpc_control_input`	M2.16	$\hat{\boldsymbol{x}}_k \to \boldsymbol{u}_0^*$ (receding horizon)
`step10_apply_to_drone`	M2.17–18	$\boldsymbol{u}_0^*$ or $-K\boldsymbol{x}_k \to \boldsymbol{\Omega}$; nonlinear flight (true state)
`step11_state_estimation_ekf`	M2.19–24	$\boldsymbol{u}_k$, pixels, gyro, $A, B \to \hat{\boldsymbol{x}}_k, \Sigma_k$; flight on $\hat{\boldsymbol{x}}_k$

A Proofs of the SO(3) identities

(2). $[\boldsymbol{a}]_\times^\top = -[\boldsymbol{a}]_\times$ by inspection of (1). $[\boldsymbol{a}]_\times \boldsymbol{a} = \boldsymbol{a} \times \boldsymbol{a} = 0$, and $[\boldsymbol{a}]_\times \boldsymbol{b} = \boldsymbol{a} \times \boldsymbol{b} = -\boldsymbol{b} \times \boldsymbol{a} = -[\boldsymbol{b}]_\times \boldsymbol{a}$.

(3). For any $\boldsymbol{b}$: $R[\boldsymbol{a}]_\times R^\top \boldsymbol{b} = R(\boldsymbol{a} \times R^\top \boldsymbol{b}) = (R\boldsymbol{a}) \times (RR^\top \boldsymbol{b}) = (R\boldsymbol{a}) \times \boldsymbol{b}$, because a rotation preserves cross products ($R(\boldsymbol{x} \times \boldsymbol{y}) = R\boldsymbol{x} \times R\boldsymbol{y}$ for $\det R = +1$).

(4). Split $M = M_s + M_a$ into symmetric and skew parts. The trace of (symmetric × skew) is zero, so $\operatorname{tr}(M[\boldsymbol{\eta}]_\times) = \operatorname{tr}(M_a[\boldsymbol{\eta}]_\times)$. With $M_a = [\boldsymbol{m}]_\times$, $\boldsymbol{m} = \frac{1}{2}(M - M^\top)^\vee$, a direct calculation gives $\operatorname{tr}([\boldsymbol{m}]_\times[\boldsymbol{\eta}]_\times) = -2\,\boldsymbol{m}^\top \boldsymbol{\eta}$. Hence $\operatorname{tr}(M[\boldsymbol{\eta}]_\times) = -\boldsymbol{\eta}^\top (M - M^\top)^\vee$.

Trace and angle. For $R = \operatorname{Exp}(\theta \boldsymbol{n})$, (5) gives $\operatorname{tr} R = 3 - (1 - \cos\theta) \cdot 2 = 1 + 2\cos\theta$ (since $\operatorname{tr}[\boldsymbol{n}]_\times = 0$ and $\operatorname{tr}[\boldsymbol{n}]_\times^2 = -2$). This is used for $\Psi = 1 - \cos\varphi$ and for Log.

B Matrix-calculus rules used

For vectors $\boldsymbol{x}, \boldsymbol{b}$ and matrices M (symmetric where stated), K, N:

$$\frac{\partial(\boldsymbol{b}^\top \boldsymbol{x})}{\partial \boldsymbol{x}} = \boldsymbol{b}, \qquad \frac{\partial(\boldsymbol{x}^\top M \boldsymbol{x})}{\partial \boldsymbol{x}} = 2M\boldsymbol{x} \ (M = M^\top),$$

$$\frac{\partial \operatorname{tr}(KN)}{\partial K} = N^\top, \qquad \frac{\partial \operatorname{tr}(KMK^\top)}{\partial K} = 2KM \ (M = M^\top).$$

The first two give the LQR gain (Model 2, block 4) and the QP gradient $H\mathbf{U} + \boldsymbol{f}$ (Model 2, block 8). The last two give the Kalman gain (Model 1, block 4).

Least squares. $\min_G \|Y - G\Omega\|_F^2$ has gradient $-2(Y - G\Omega)\Omega^\top$. It is zero at $G = Y\Omega^\top(\Omega\Omega^\top)^{-1} = Y\Omega^\dagger$ when Ω has full row rank (DMDc, Model 2 block 2).

C References

[1] T. Lee, M. Leok, N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)", *IEEE CDC*, 2010 (arXiv:1003.2005). Basis of Model 1, blocks 5–7.

[2] J. Solà, "Quaternion kinematics for the error-state Kalman filter", arXiv:1711.02508, 2017. Basis of Model 1, block 4.

[3] J. L. Proctor, S. L. Brunton, J. N. Kutz, "Dynamic mode decomposition with control", *SIAM J. Applied Dynamical Systems* 15(1), 2016. Model 2, block 2.

[4] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, Vol. I, Athena Scientific. Model 2, blocks 3–6.

[5] J. B. Rawlings, D. Q. Mayne, M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed., 2017. Model 2, blocks 7–9.

[6] T. Qin, P. Li, S. Shen, "VINS-Mono: a robust and versatile monocular visual-inertial state estimator", *IEEE T-RO*, 2018.

[7] ArduPilot SITL and Gazebo Iris model documentation; project parameters from `controller.yaml` (package `drones_controller`, folder `config`).

D Glossary

Term	Meaning
ArduPilot SITL	ArduPilot flight software running on a PC (Software-In-The-Loop), flying a simulated drone
Attitude	orientation of the drone; here the rotation matrix R
Body frame	axes fixed to the drone: x forward, y left, z up (FLU)
Covariance	matrix of variances/co-variations of errors; the EKF's "how unsure am I"
DARE	Discrete Algebraic Riccati Equation; gives the infinite-horizon LQR solution
DMDc	Dynamic Mode Decomposition with control; fits $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$ to data by least squares
EKF	Extended Kalman Filter: Kalman filter applied to a nonlinear model through linearization
ENU	East–North–Up world frame used by ArduPilot's ROS 2 topics
Error-state	the filter tracks the small error $\delta\boldsymbol{x}$, not the full state, so attitude is handled correctly
Gazebo	3D physics simulator in which the Iris quadrotor flies
Gimbal lock	singularity of Euler angles at $\pm 90°$ pitch; avoided by working with R directly
IMU	Inertial Measurement Unit: gyroscope (rates) and accelerometer
Innovation	measurement minus what the filter expected to measure
Jacobian	matrix of partial derivatives; the "slope" of a vector function
Kalman gain K	how strongly the estimate is pulled towards a measurement
LQR	Linear Quadratic Regulator: optimal linear feedback $\boldsymbol{u} = -K\boldsymbol{x}$
Lyapunov function	energy-like quantity that never increases; proves stability
MPC	Model Predictive Control: repeatedly solve a finite-horizon optimal problem with constraints
QP	Quadratic Program: minimise a quadratic cost subject to linear constraints
Receding horizon	apply only the first planned input, then re-plan one step later
RK4	4th-order Runge–Kutta: accurate way to step a differential equation forward
ROS 2	Robot Operating System; our controller node talks to ArduPilot through it
SO(3)	the set of all 3D rotation matrices ($R^\top R = I$, $\det R = 1$)
VIO	Visual-Inertial Odometry: estimating motion by fusing camera and IMU
ZOH	Zero-Order Hold: input kept constant between samples